LONDON METROPOLITAN UNIVERSITY

ITAHARI INTERNATIONAL COLLEGE

Module Code & Module Title

CS4001NT Programming

Year and Semester

2025 Spring, 2nd Semester

Student Name: Darshana Bhandari

London Met ID: 24045861

College ID:NP05CP4A240206

Assignment Due Date: 29 July

Assignment Submission Date: 1 August

Submitted To: Sujan Tamang

Word Count (Where Required):11546

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Darshana Bhandari

Islington College,Nepal

Document Details

Submission ID

trn:oid:::3618:106298277

Submission Date

Jul 29, 2025, 3:03 PM GMT+5:45

Download Date

Jul 29, 2025, 3:06 PM GMT+5:45

File Name

Darshana Bhandari

File Size

34.4 KB

38 Pages

5,278 Words

27,047 Characters

4% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **25** Not Cited or Quoted 4%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
0% Publications
4% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **25** Not Cited or Quoted 4%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
0% Publications
4% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	
	islingtoncollege on 2025-05-23	3%
2	Submitted works	
	Informatics Education Limited on 2010-03-17	<1%
3	Submitted works	
	Nottingham Trent University on 2024-08-18	<1%
4	Submitted works	
	Heriot-Watt University on 2024-10-29	<1%
5	Submitted works	
	University of Greenwich on 2009-12-11	<1%

Table of Contents

1. Introduction 1

1.1 Introduction to coursework 1

1.2 Aims and objective 1

1.3 Java 2

1.4 Bluej 2

1.4 Wire frame 3

1.4 Google Docs 4

2. GUI 4

3. Class Diagram 5

3.1 GYM Member Class Diagram: 6

3.2 Regular member class Diagram 7

3.3 Premium Member Class Diagram: 7

3.4 Combined Class Diagram: 8

4. Pseudocode 9

4.1 File Operations 9

4.2 Member Management 10

4.3 Membership Operations 11

4.4 Plan & Payment Handling 12

4.5 Reverting Members 13

4.6 Display & Utilities 14

5. Method Description 15

5.1 Gym GUI Method and Description 15

5.2 File operation method 17

5.3 Gym Member Method and Description 19
5.3 Regular Member Method & Description 19
5.4 Premium Member Method & Description 20
6. Testing 20
6.1 Test - 1 : Compile and Running Using Command Prompt or Terminal. 20
6.2 Test-2: Add Regular and Premium Members 21
6.2.1 Regular Member 21
6.2.2 Premium member 23
6.3 Test-3: Mark Attendance and Upgrade 24
6.3.1 Mark Attendance 24
6.3.2 Upgrade Plan 25
6.4 Test-4: Calculate Discount, Pay Due, Revert Member 26
6.4.1 Calculate Discount 26
6.4.2 Pay Due Amount 27
6.5.1 Save to file 30
7.1 Syntax Error 32
7.3 Runtime Error 34
7. Conclusion 34
8.1 Evaluation of Work 35
8.2 Reflection and Learn 35
8.3 Difficulties Face 36
8.4 How Difficulties Were Overcome 36
9 References 37
9.1 file:///C:/Users/sp/Desktop/java.htm 37
9.2 file:///C:/Users/sp/Desktop/File_BlueJ_Logo.svg 37

9.3 file:///C:/Users/sp/Desktop/File_Google_Docs_2020_Logo.svg 37

..... 37

10. Appendix 38

10.1 Code Listing for GYM Member 38

10.2 code listing for Regular Member 43

10.3 code Listing for Premium Member 49

10.4 Code Listing for GYM GUI 57

Table of Figure

Figure 1 : Java logo 2
Figure 2 : Bluej Logo 2
Figure 3 : Wire-frame 3
Figure 4 : Google docs Logo 4
Figure 5 : sample GUI 5
Figure 6 : Class Diagram 6
Figure 7 : GYM Member class Diagram 6
Figure 8 : Regular member class Diagram 7
Figure 9 : Premium Member class diagram 7
Figure 10 : comboned Class Diagram 8
Figure 11 : complaing using terminal 21
Figure 12 : Regular Member successful Button testing 22
Figure 13 : Premium member successful Button Testing 24
Figure 14 : Mark Attendance sucessful testing 25
Figure 15 : Upgrade Button sucessful test 26
Figure 16 : calculate Discount Button Sucessful test. 27
Figure 17 : Pay Due Amount Sucessful test. 28
Figure 18 : Revert Member Button Successful Test 29
Figure 19 : save to file sucessful test. 30
Figure 20 : Read from file Button successful test. 31
Figure 21 : Syntax Error Correction & Decatio 33
Figure 22 : Logical error Correction & Decation. 33
Figure 23 : Runtime Error Correction & Decation 34

Table of Tables

Table 1 : GYM GUI Method&Description 15
Table 2 : Savetofile & Readfromfile 17
Table 3 : GYM member Method & Description 19
Table 4 : Regular Member method & Description 19
Table 5 : Premium Member Method & Description 20
Table 6 : Compile and Running Using Command Prompt or Terminal. 20
Table 7 : Regulur Member button testing 21
Table 8 : Premium Button Testing 23
Table 9 : Mark Attendance Button testing 24
Table 10 : upgrade plan Button testing 25
Table 11 : calculate Discount Button testing 26
Table 12 : pay due Amount Testing 27
Table 13 : Revert Member Button Testing 28
Table 14 : save to File button Testing 30
Table 15 : Read from File Button Testing 31

1. Introduction

1.1 Introduction to coursework

The purpose of this coursework is to design a gym management system using object oriented programming(oop). The goal is to develop a system that can handle various tasks in a gym. Such as, managing membership, tracking attendance, processing payment and more. This system will automate several aspects of gym management making it easier for gym owners and managers to cover memberships, track payment, schedule classes, and manage trainers and equipment. This course work required three core classes: Gym member, Regular member, and premium member, where premium members were sub classes of the abstract parent classes Gym member. Each member will have unique attributes like ID, Name, Membership, Start date and attendance record.

This coursework is divided into two milestones. The first milestone is focusing on oop implementation and creation of the GUI. While the second milestone includes event handling and exception handling. In simple words, we'll be learning how to create a system that manages gym membership. This project will give us the chance to practice writing code, solve real world problems and develop our programming skills. It's also a great way to learn how to create useful programs that people can actually use.

1.2 Aims and objective

These are main aims and objective of this coursework:

i. To develop our problems solving skills, knowledge and ideas.
ii. To give us hands on experience with real world application of oop and software development practice.
iii. To make us more familiar with java programming and gain more practical experiences.
iv. To Teach us more about coding and system design.

v. To make the system easy to use, secure and able to handle all the gyms needs.
vi. To make us learn about good programming practice on coding.
vii. To make a program that is easy to use and run smoothly.

1.3 Java

Figure 1: Java logo

Java is a computer programming language which is one of the languages used in online distribution outlets. Java is a simple and easy to understand high-level language. Java is a broad application in embedded system designing, Consul GUIs application, Web and mobile games application. Java is also used to create device software. Java is everywhere, Write once a program. The code would be just written once only, yet it would be applicable everywhere. Java files Java are interpreted on the Java interpreter which processes them after they have been translated into bit code. Java is multi thread capable. Java can multitask because of multi-threading functionality. Java is portable since it has the capability to run on any platform and compatibility with data types whose sizes are known

1.4 Bluej

Figure 2 : Bluej Logo

Java programs are written and coded with a software application(Blue J). It is chiefly focused on students and people with no experience who have never gotten into programming. The design of Blue J is extremely simple and easy to understand and hence the learner is able to focus more on the learning of the basic things regarding Java especially object oriented programming. Working with Blue J, it will be easier to understand the principle of the functioning of a program since it is possible to create classes and objects and display them on the screen. One

can also test whether the small snippets of codes go and identify the errors and start programs within the same window as well. Schools and colleges have to some extent made use of Blue J as a teaching program in Java programming, but as it is made far too useful to be used at all schools and colleges, so easy to use.

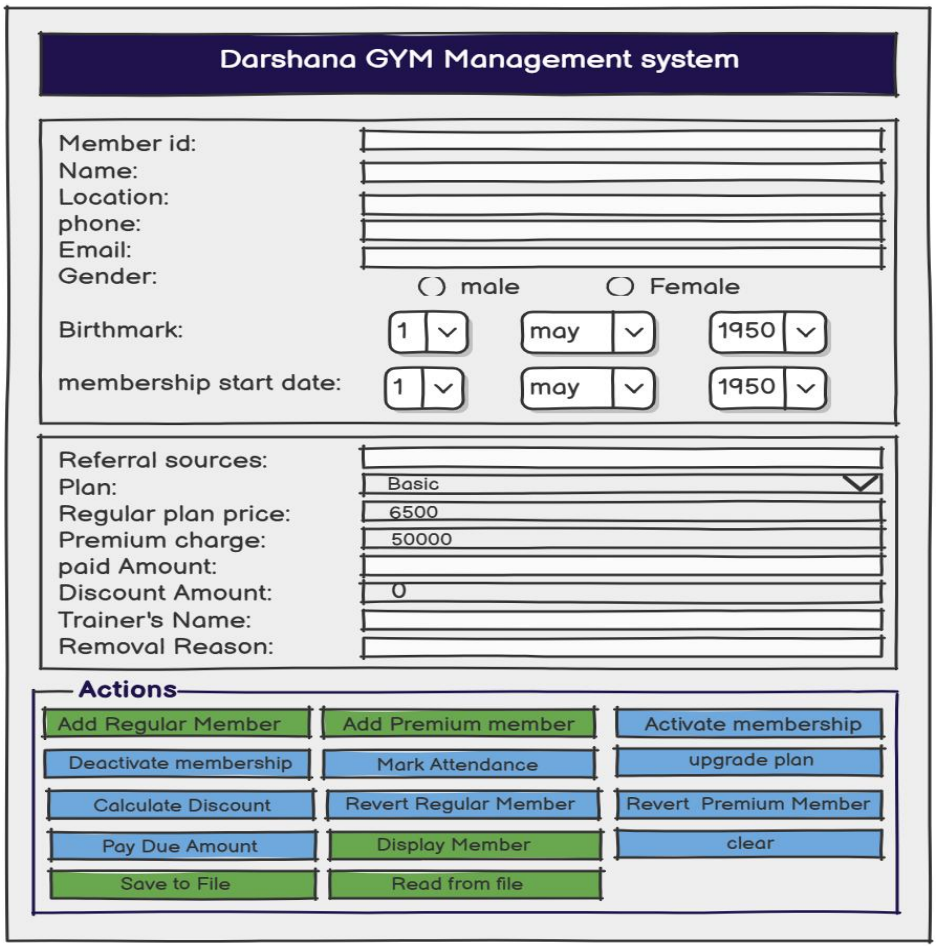
1.4 Wire frame

Wire-frame is a sketch outline or visual blueprint of our website before any kind of design elements or development even takes place. Wire-frame shows the basic structure where things like menu, image, text, buttons and other features will go.

Wire-frames save our time, money and focus on function not style. There are many tools like sketch, Balsamic etc. That helps us to create wire-frames, some of which are very quick, where others are more detailed. A wire frame is like a map of a website or app.

It's basically a visual representation of elements on a website. It consists of horizontal or vertical lines that form boxes denoting contains that further contain different elements of the web-page.

Figure 3 : Wire-frame



1.4 Google Docs

Figure 4 : Google docs Logo

Google Docs refers to the online word-processing application that Google developed and this application allows them to write, and edit documents in a text format through a web browser. It is also cloud based, i.e. whatever you will be doing, it will automatically save in Google drive since you will be able to access such things comfortably on any machine. Real-time collaboration is one of the most useful tools it offers because it provides several users with an opportunity to work on the same document simultaneously and see the changes in real time. The document is also fantastic with regard to collaborative work of any kind or any other professional purpose since the user can comment, suggest some editing and even chat. It supports various types of documents such as. docx, .pdf and. txt, and import/export of documents is simple. Google docs also has other templates to select including resumes, letters and reports to get started within a short time. The document's backup occurs automatically and the history of versions gives an opportunity to trace the changes and go back to the previous version at any time. In case you want to make some offline changes, you should turn on the offline mode and then make changes once you are back online. It is easy to share with and you control who gets to access your document whether to view/comment on or even get to edit your piece of work. Overall, it is one of the most powerful, convenient personal and academic, even business writing tools.

2. GUI

A graphical user interface (GUI) is the portion of an operating system or software that you see and interact with using text fields, menus, windows, buttons, and icons rather than inputting instructions.

Real-world example:

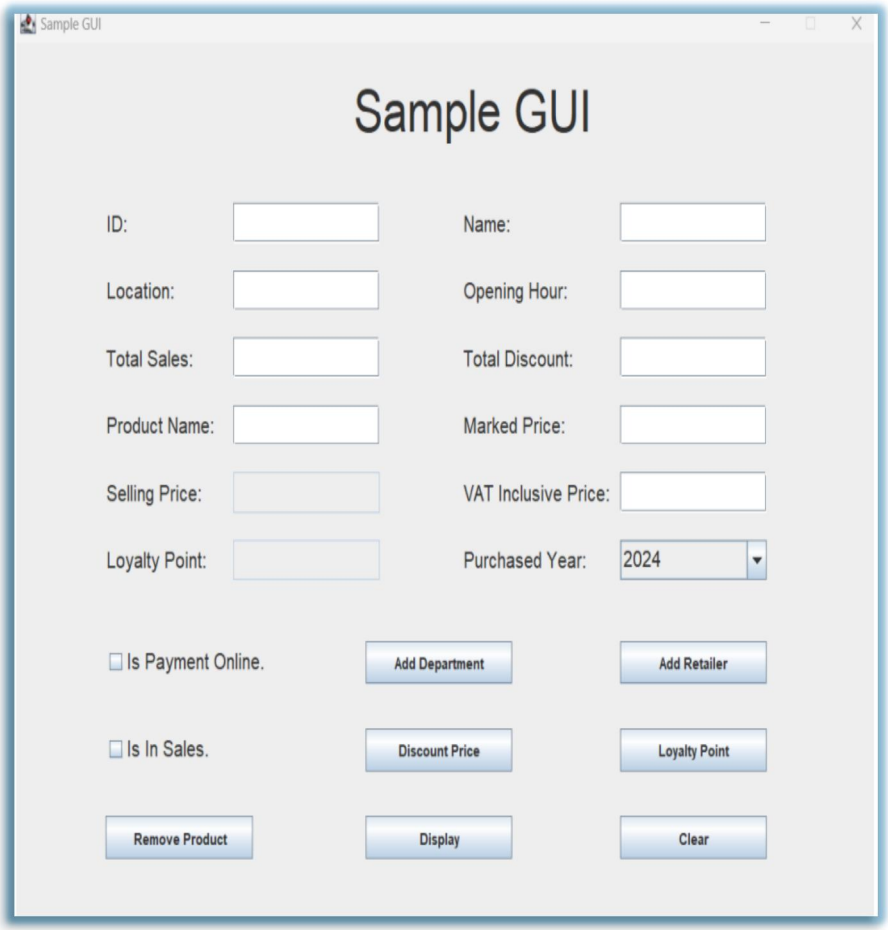
When you launch a mobile application and press a button, that's a graphical user interface.

The "Start" menu in Windows is a graphical user interface.

One example of a GUI is when you fill out a form on a website.

Gui is crucial because due to the fact that it makes software user-friendly, especially for those without coding knowledge.

Figure 5 : sample GUI

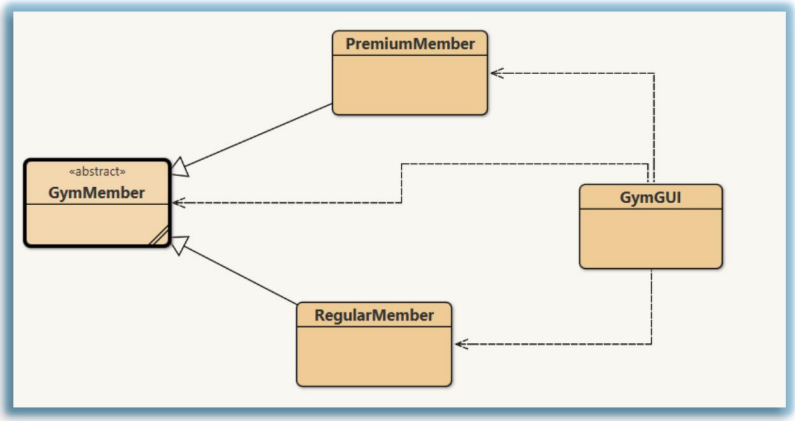


3. Class Diagram

A class diagram is simple to show the structure of a computer program by using classes. Everything in object-oriented programming is built from classes, and a class diagram displays those classes openly. Each box on the diagram signifies a product, a customer,

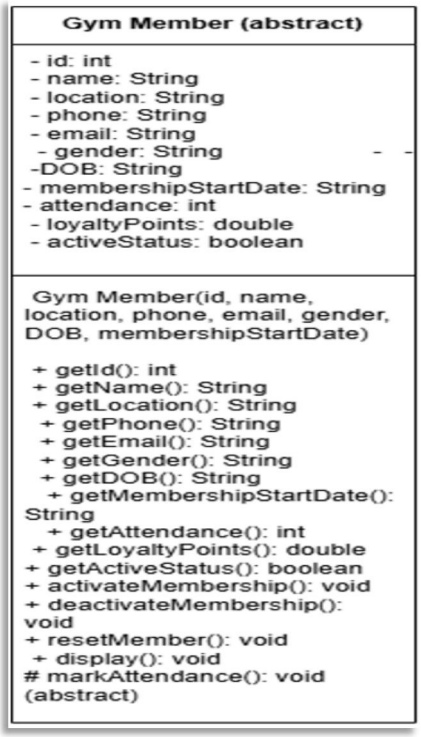
or an invoice, things that actually exist. The classes within the diagram have their names and the kinds of data they contain (like a product's details) and what they are able to do like calculating costs or updating inventory).It demonstrates the relationship among the classes. An example is that one class can inherit from another class, or it can be based on a previous class, or it might just add parts to the same functionality. Class diagrams are useful as they explain in advance how the system will be created without code. It's similar to designing a detailed outline for your program so that you know how each section fits in. This helps a lot on group assignments or big projects, since members of the team can all see how the system is organized.

Figure 6 : Class Diagram



3.1 GYM Member Class Diagram:

Figure 7 : GYM Member class Diagram



3.2 Regular member class Diagram

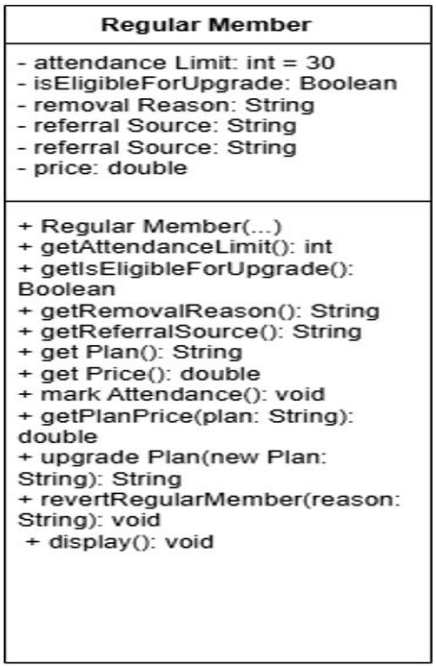
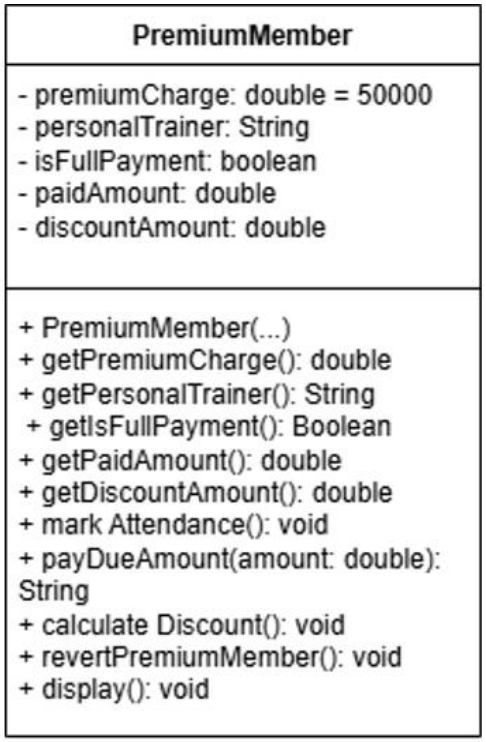


Figure 8 : Regular member class Diagram

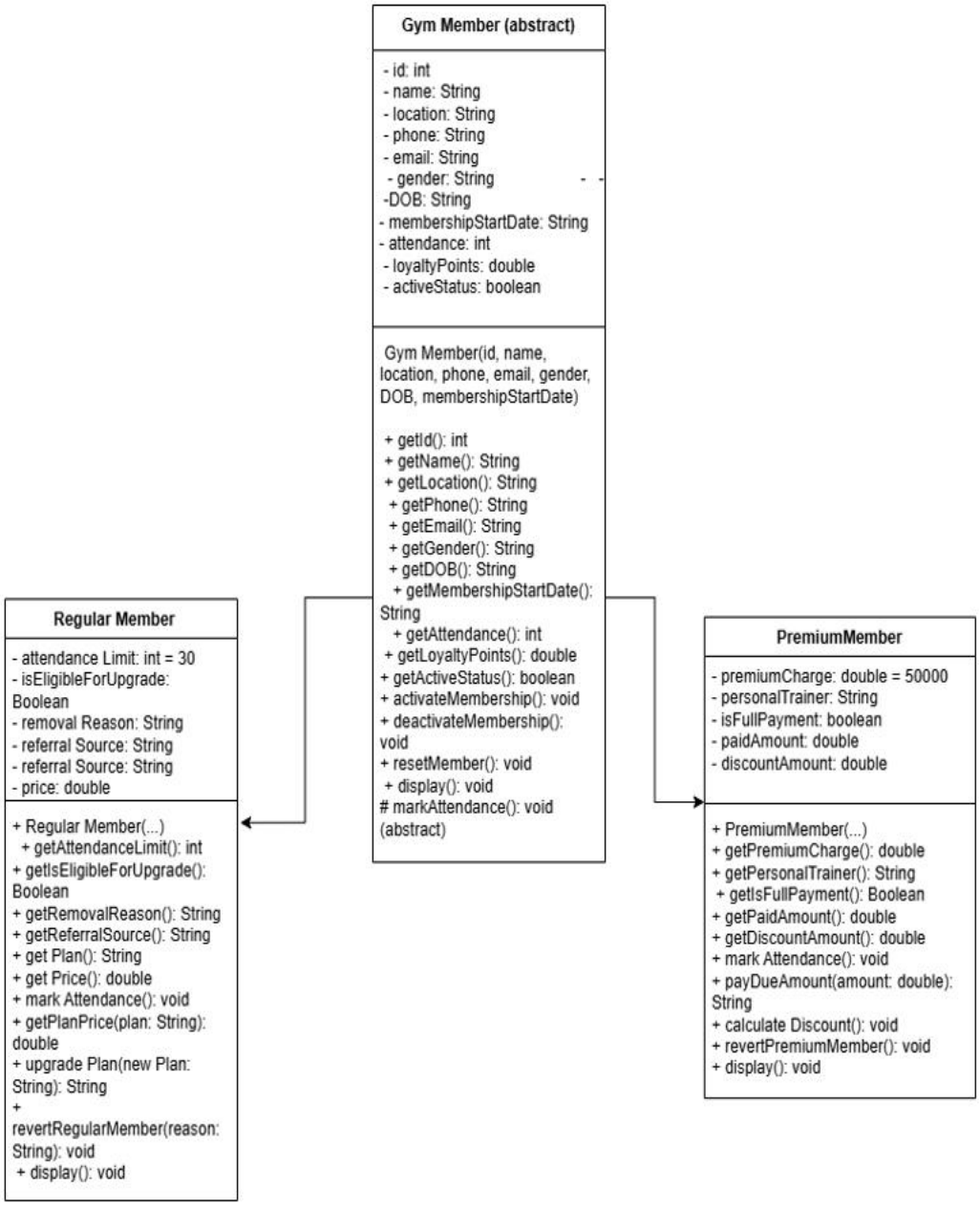
3.3 Premium Member Class Diagram:

Figure 9 : Premium Member class diagram



3.4 Combined Class Diagram:

Figure 10 : comboned Class Diagram



4. Pseudocode

4.1 File Operations

```
METHOD saveToFile():
    TRY
        OPEN "MemberDetails.txt" in write mode
        WRITE formatted header (ID, Name, Location, etc.)
        FOR EACH member IN members:
            IF member IS RegularMember:
                SET plan = member.getPlan()
                SET price = member.getPrice()
                SET paymentInfo = "N/A"
            ELSE IF member IS PremiumMember:
                SET plan = "Premium"
                SET price = member.getPremiumCharge()
                SET paymentInfo = member.getPaymentStatus()
            WRITE formatted member data
        SHOW "Data saved successfully" message
    CATCH IOException:
        SHOW "Error saving file" message
END METHOD


METHOD readFromFile():
    IF "MemberDetails.txt" does not exist:
        SHOW "File not found" error
        RETURN
    TRY
        OPEN file in read mode
        READ all lines into a string buffer
```

```
        CREATE a scroll able text area with mono spaced font
        DISPLAY content in a dialog box
    CATCH Io Exception:
        SHOW "Error reading file" message
END METHOD
```

4.2 Member Management

```
METHOD addRegularMember():
    TRY
        id = PARSE_INT(idField.text)
        IF isIdDuplicate(id):
            SHOW "ID already exists" error
            RETURN
        VALIDATE all required fields (name, location, etc.)
        IF any field is empty:
            SHOW "Fill all fields" error
            RETURN
        CREATE new RegularMember with input data
        ADD to members list
        SHOW "Member added" success
        CLEAR input fields
    CATCH Number Format Exception:
        SHOW "Invalid ID" error
END METHOD

METHOD addPremiumMember():
    (Same structure as addRegularMember but creates PremiumMember)
END METHOD
```

```
METHOD is Id Duplicate(id):
   FOR EACH member IN members:
      IF member.id == id:
         RETURN true
   RETURN false
END METHOD
```

4.3 Membership Operations

```
METHOD activateMembership():
   TRY
      id = PARSE_INT(idField.text)
      member = FIND member WITH id
      IF member FOUND:
         CALL member.activate Membership()
         SHOW "Activated" success
      ELSE:
         SHOW "Member not found" error
   CATCH Number Format Exception:
      SHOW "Invalid ID" error
END METHOD

METHOD deactivateMembership():
   (Same as activateMembership but calls deactivate)
END METHOD

METHOD markAttendance():
```

```
    TRY
      id = PARSE_INT(idField.text)
      member = FIND member WITH id
      IF member FOUND:
        IF member.isActive():
          CALL member.markAttendance()
          SHOW "Attendance marked"
        ELSE:
          SHOW "Inactive member" warning
      ELSE:
        SHOW "Member not found" error
    CATCH Number Format Exception:
      SHOW "Invalid ID" error
  END METHOD
```

4.4 Plan & Payment Handling

```
METHOD upgradePlan():
  TRY
    id = PARSE_INT(idField.text)
    plan = GET_SELECTED_PLAN()
    member = FIND member WITH id
    IF member FOUND AND member IS RegularMember:
      IF member.isActive():
        result = CALL member.upgradePlan(plan)
        SHOW result
      ELSE:
        SHOW "Inactive member" warning
    ELSE:
      SHOW "Not a Regular Member" error
  CATCH Number Format Exception:
    SHOW "Invalid ID" error
```

```
END METHOD

METHOD calculate Discount():
   TRY
      id = PARSE_INT(idField.text)
      member = FIND member WITH id
      IF member FOUND AND member IS PremiumMember:
         CALL member.calculate-discount()
         UPDATE discountField
         SHOW "Discount calculated"
      ELSE:
         SHOW "Not a Premium Member" error
   CATCH Number Format Exception:
      SHOW "Invalid ID" error
END METHOD

METHOD payDueAmount():
   TRY
      id = PARSE_INT(idField.text)
      amount = PARSE_DOUBLE(paidAmountField.text)
      member = FIND member WITH id
      IF member FOUND AND member IS PremiumMember:
         result = CALL member.pay Due Amount(amount)
         SHOW result
      ELSE:
         SHOW "Not a Premium Member" error
   CATCH Number Format Exception:
      SHOW "Invalid input" error
END METHOD
```

4.5 Reverting Members

```
METHOD revertRegularMember():
   TRY
      id = PARSE_INT(idField.text)
      reason = removal Reason Field.text
      IF reason IS EMPTY:
         SHOW "Enter reason" warning
         RETURN
      member = FIND member WITH id
      IF member FOUND AND member IS RegularMember:
         CALL member.revert Regular Member(reason)
         SHOW "Reverted" success
      ELSE:
         SHOW "Not a Regular Member" error
   CATCH Number Format Exception:
      SHOW "Invalid ID" error
END METHOD


METHOD revertPremiumMember():
   (Same as revert Regular Member but for Premium Member)
END METHOD
```

4.6 Display & Utilities

```
METHOD displayMembers():
   CREATE new window
   IF members list IS EMPTY:
      SHOW "No members" message
   ELSE:
      FOR EACH member:
         IF member IS RegularMember:
            APPEND "REGULAR MEMBER" header
         ELSE:
```

```
            APPEND "PREMIUM MEMBER" header
        APPEND formatted member details
    DISPLAY in scroll able text area
END METHOD


METHOD clearFields():
    RESET all text fields to ""
    RESET combo boxes to first item
    RESET radio buttons to unselected
    SET discount Field to "0"
END METHOD
```

5. Method Description

A method description explains, in writing, what the function of a method is in the program. It explains the goal, the way the method works and how it is supposed to act. It gives a reader an idea of understanding what data is used, what result it is going to give, and how it works with a program. Here, is a description of method description that are used in my Darshanagymmanagementsystem:

5.1 Gym GUI Method and Description

Table 1: GYM GUI Method&Description

Method	Description
style Button(),style TextField), styleLabel(),styleRadioButtonn(), stylePanel()	utility techniques that help all UI parts feel and look the same by having a consistent design
design.setupUI()	Every button, combo box, input field, and panel is set up and initialized, and all the elements are

	put in the layout with the same design
setupDate ComboBoxes()	Makes it possible for users to choose a birth date and the day their membership started using combo boxes.
Activate Membership(), deactivate Membership()	use ID to find a member and update their active status
markAttendance()	Registers that a member was present, and increases their attendance and loyalty
Upgrade Plan()	Promotes the plan of a Regular Member to a more advanced plan and changes the price.
revertRegularMember(), revertPremiumMember()	changes the member status and takes action based on their type.
payDueAmount()	Process payments of PremiumMember and makes their status to be paid in case the total payment is made
display Members()	View of all members in the system and only the specific fields required by each member type can be viewed
clear Fields()	Then resets the input fields to the initial position.
addRegularMember()	Checks each input, ensures there is no duplicate ID, makes a new Regular Member with the entered information, and puts it in the members list.
addPremiumMember()	function adds details from the new member to a new Premium Member object, adds the object to the list of members, checks the input fields,

	and checks for duplicate IDs.
isidDuplicate()	Guarantees that new member IDS are not already in the system during the time.
Calculate discount()	This method is used to calculate discounts for Premium Member relying on their attendance and how loyal They are: points.

5.2 File operation method

Table 2: Savetofile & Readfromfile

Method	Description
SavetoFile();	The saveToFile procedure copies the information of the current object to a file in the filesystem. This method has only a single parameter, file name, of String type and represents the place where data should be stored and does not professionally return value. The approach will change the current state of an object to a long-term storage as follows: an object will be opened or created in the specified file and the data will be converted to the appropriate form of storage (text, binary, or serialized) and the re-formatted data will be written in the file. This is done to ensure effective management of the resources in that the resources in the system are free once a write operation has been performed so as to prevent corruption of data. It also has error

	handling which is used to deal with file access issues; and the usual thing that occurs when there are issues accessing, creating or writing to the file is that an Io Exception is thrown.
ReadtoFile();	The read From File method will extract the information contained in a particular file and will populate the attributes in the current object with the details. It only takes a parameter whose data type is String and is called filename used to put in a filename of the file to be open and has void data type because it does not give a value just as it only modifies the state of the current object. The process involves DE-serializing information which has already been written by opening the specified file, interpreting and reading the information in it and processing such bits of information to there and back the object properties again in order to restore the state of the object. It also entails the process of closing the file where we read so as to be able to release the system resources. The method occurs to check the error, and in various failure conditions; the method tends to either raise Io-exception in the instance of file access or File Not Found Exception in the instance the file is not located. The data format of the file must align with that of the save To File method, such that, the information can be reassembled correctly.

5.3 Gym Member Method and Description

Table 3 : GYM member Method & Description

Method	Description
activateMembership()	Makes the member's status active.
deactivateMembership()	Makes the member's status inactive.
markAttendance()	Adds one to the attendance count and loyalty points each time a member comes to the gym.

5.3 Regular Member Method & Description

Table 4 : Regular Member method & Description

Methods	Description
upgradePlan()	If the member uses upgrade Plan the plan is upgraded to a higher tier, and the price is changed too. You will receive a message stating whether the requested plan is available or if you are already part of it.
Revet MemberRegular()	Ends membership for the specified member and lists the reason in the database

5.4 Premium Member Method & Description

Table 5 : Premium Member Method & Description

Methods	Description
Calculates discount()	Knowing how many points attendants have gained, discount calculates the amount to be deducted from their fee, but only when full payment is received.
Payment due()	the payment amount is subtracted from the premium charge, the payment status is changed if all is paid, and a result message is given back
Revert premium member()	returns a premium member to having no payment made, having paid nothing, and having the initial discount

6. Testing

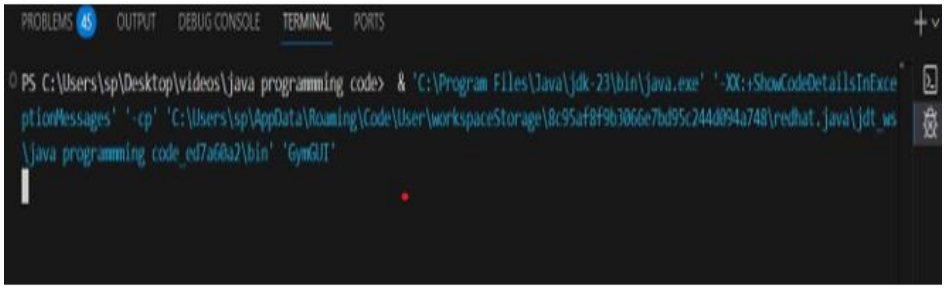
6.1 Test - 1 : Compile and Running Using Command Prompt or Terminal.

Table 6 : Compile and Running Using Command Prompt or Terminal.

Objective	to use the command prompt for program compilation and execution.
Action	Use the command java Gym GUI to build the

	application after opening the command prompt from the file location, Java and executed with the java gym GUI command
Expected Output	The file should correctly build, execute, and show the result graphical user interface.
Actual Output	The GUI opened when the software had finished compiling and running.
Result	The test worked well.

Figure 11 : complaing using terminal



6.2 Test-2: Add Regular and Premium Members

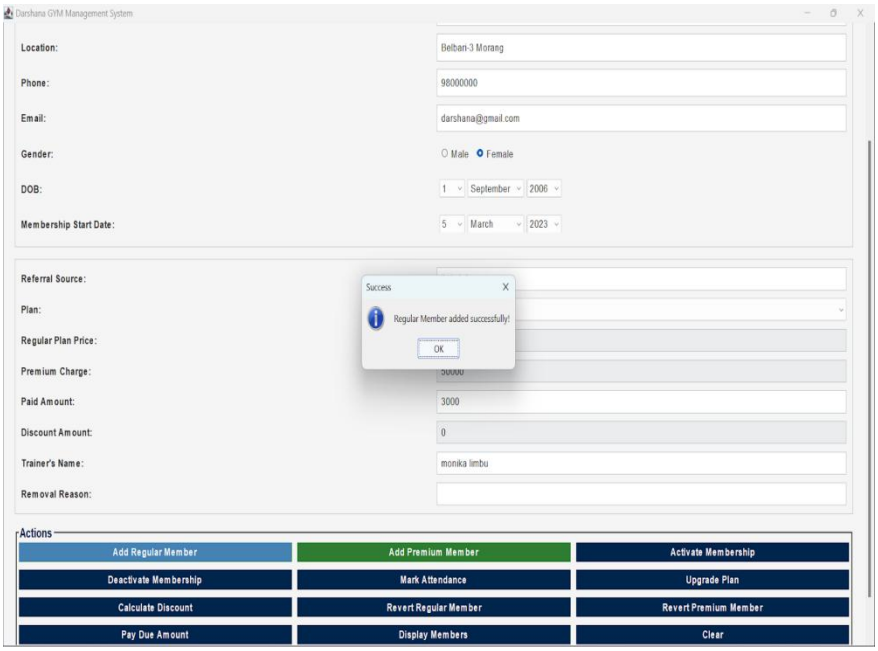
6.2.1 Regular Member

Table 7 : Regulur Member button testing

Objective	Including a Regular Member.
Action	Lunch gym GUI and input the information Member id: 1 Name: Darshana Bhandari

	Location: Belbari-3 Morange Email: darshana@gmail.com Gender: Female Date of Birth: 01/09/2006 Membership start Date: 05/03/2023 Referral Sources: hwjgdghsdh Paid Amount: 3000 Trainer name: monika limbu
Expected Output	The addition of regular members should be successfully.
Actual Output	Regular members added successfully.
Result	It was a successful test.

Figure 12 : Regular Member successful Button testing

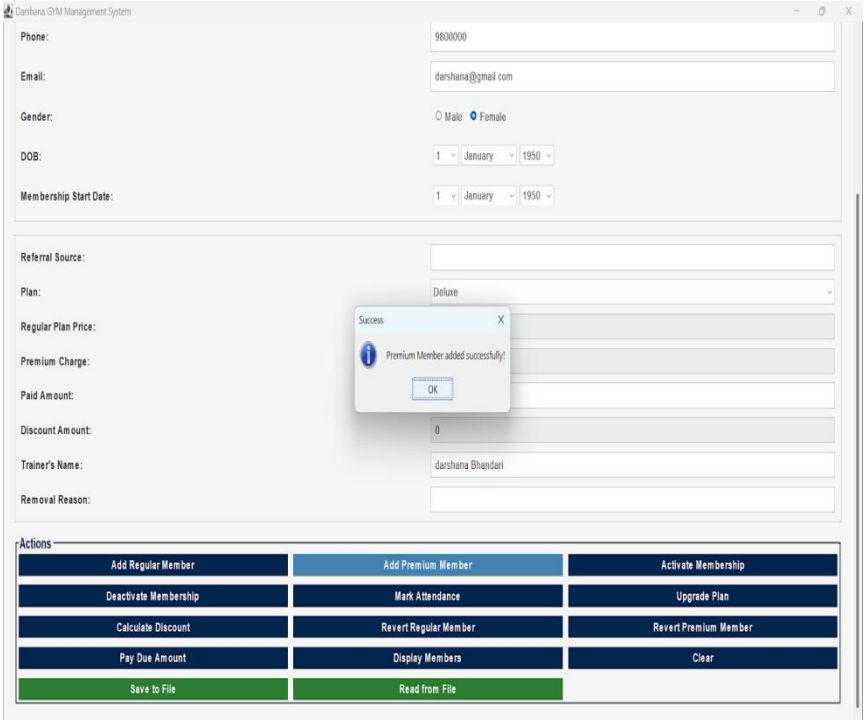


6.2.2 Premium member

Table 8 : Premium Button Testing

Objective	Including Premium Member
Action	Lunch gym GUI and input the information Member id: 2 Name: Monika Limbu Location: Belbari-3 Morange Email: darshana@gmail.com Gender: Female Date of Birth: 01/09/2006 Membership start Date: 05/03/2023 Referral Sources: hwjgdghsdh Paid Amount: 4000 Trainer name: Darshana Bhandari
Expected Output	The addition of premium members should be successful.
Actual Output	premium members added successfully.
Results	It was a successful test.

Figure 13 : Premium member successful Button Testing



6.3 Test-3: Mark Attendance and Upgrade

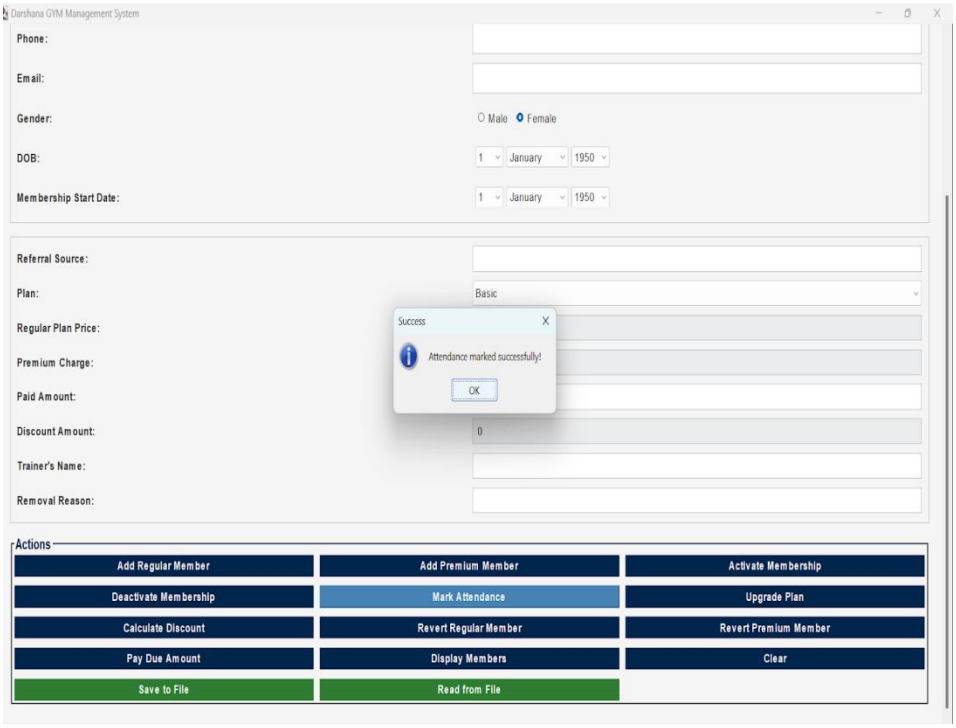
6.3.1 Mark Attendance

Table 9 : Mark Attendance Button testing

Objective	The Attendance Mark Button is Being Tested.
Action	You must first enter the Id of a member in the dialog box, and you have to click on the mark attendance button.
Excepted output	The window should show your mark attendance is done or successful.
Actual output	It should display attendance marked successfully.

Result	The test was successful.

Figure 14 : Mark Attendance sucessful testing



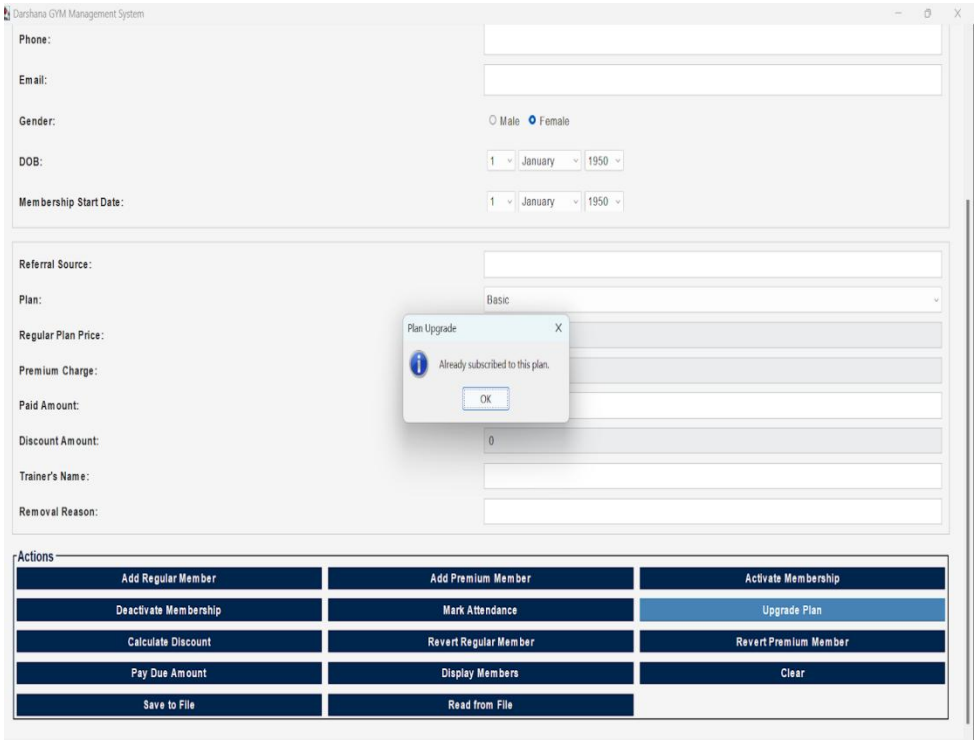
6.3.2 Upgrade Plan

Table 10 : upgrade plan Button testing

Object	The upgrade plan button is being tested.
Action	You can upgrade plans only if you are a regular member, to upgrade a plan you should have a proper attendance first then only you can upgrade. At first , you need to add a member id in the dialog box then you have to click on the upgrade plan button .
Excepted Output	The window should display successful if the upgrade plan is done or successful.
Actual Output	It should display Already subscribe to this plan.

Result	The test was successful.

Figure 15 : Upgrade Button sucessful test



6.4 Test-4: Calculate Discount, Pay Due, Revert Member

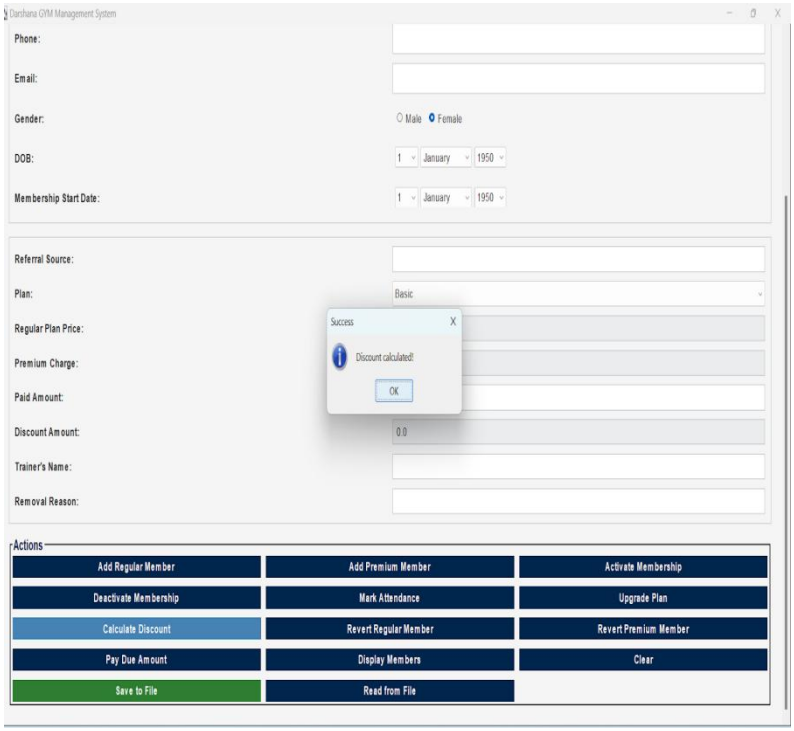
6.4.1 Calculate Discount

Table 11 : calculate Discount Button testing

Objective	The Calculate Discount button is being Tested.
Action	You have to add a member id number in the member id dialog box. And also You have to make a full payment to get a discount and after that you have to click on calculate discount Button.
Excepted Output	The window should display successful discount if discount

	is have been calculated
Actual Output	It will display a discount calculated and you have to click on the OK button.
Result	The test was successful.

Figure 16 : calculate Discount Button Sucessful test.



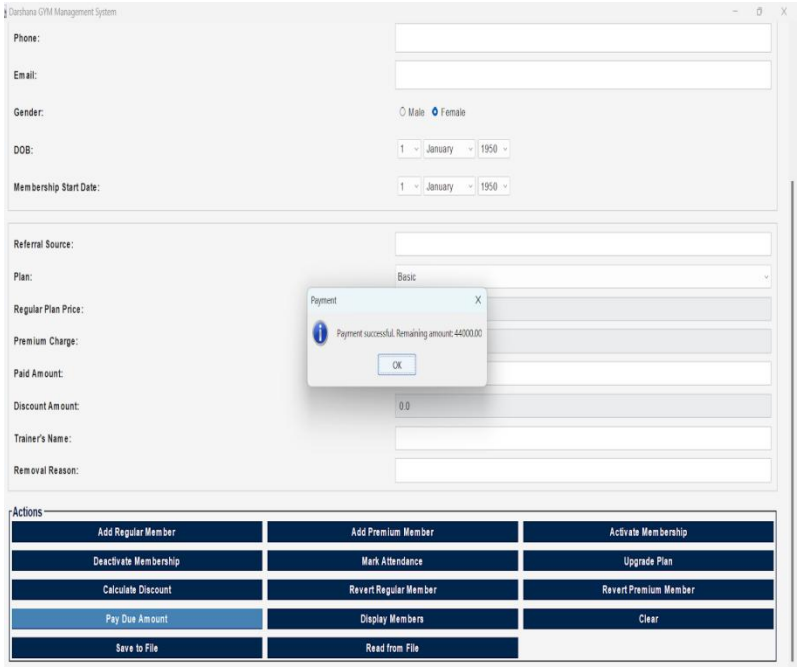
6.4.2 Pay Due Amount

Table 12 : pay due Amount Testing

Objective	The pay Due button is being tested.
Action	You have to enter the member id number in the member id field of the dialog box, and you have to also enter your payment in the pay Due amount field of the dialog box. After that you have to click on the pay due amount Button.

Expected output	The window should display a successful pay due amount.
Actual output	It will display payment successful output. And if you have not paid the full Amount it will also display your remaining amount to pay and click on OK button.
Result	The test was successful .

Figure 17 : Pay Due Amount Sucessful test.



6.4.3 Revert Member

Table 13 : Revert Member Button Testing

Objective	The Revert member button is being tested.

Action	There is two button revert member button one is for regular member and another is for premium member to use revert member button first you have to enter member id in member id field of the dialog box. Then click a revert button for the type of member you want to revert.
Expected Output	The window should display if the revert member is successful.
Actual Output	The message will display according to your revert if you are reverting a premium member it will display “premium member reverted successfully” and if you are reverting regular member revert it will display “regular member reverted successfully” and after that you must click the ok button.
Result	The test was successful.

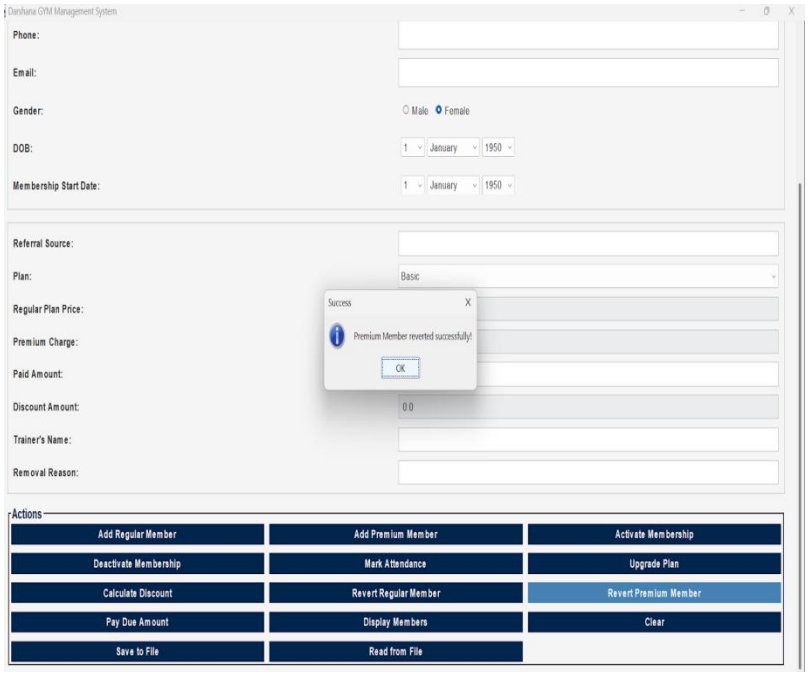


Figure 18 : Revert Member Button Successful Test

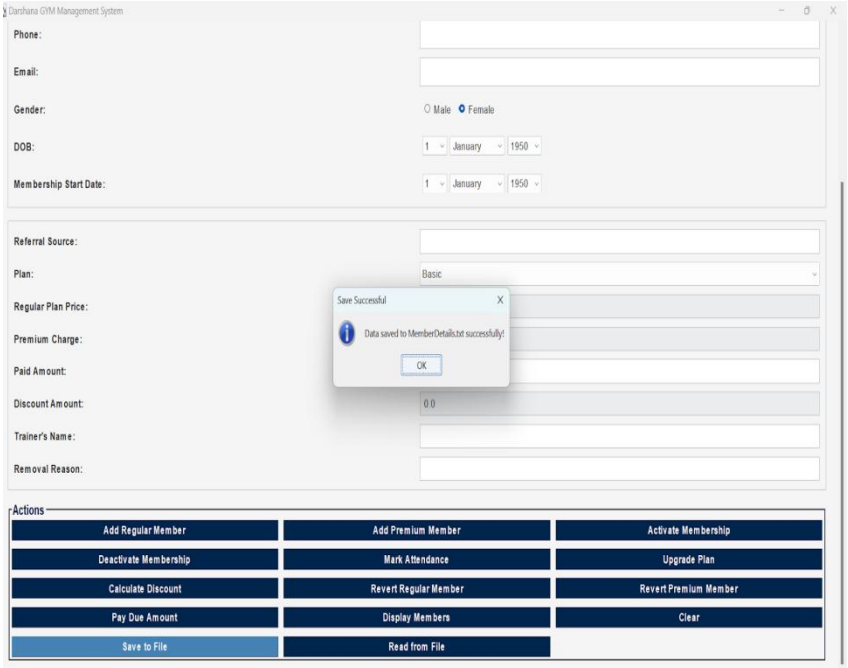
6.5 Test-5: Save to and Read from file

6.5.1 Save to file

Table 14 : save to File button Testing

Objective	The save to file button is being tested.
Action	You must have to enter the member id in the member id field of the dialogue box. And after that click the “save to file “button to save members data. member data also can be saved when you are adding a regular member or premium member.
Excepted Output	The window should display successful output.
Actual output	The message will be displayed “ data saved to memberDetails.txt successfully”.
Result	The test was successful.

Figure 19 : save to file sucessful test.

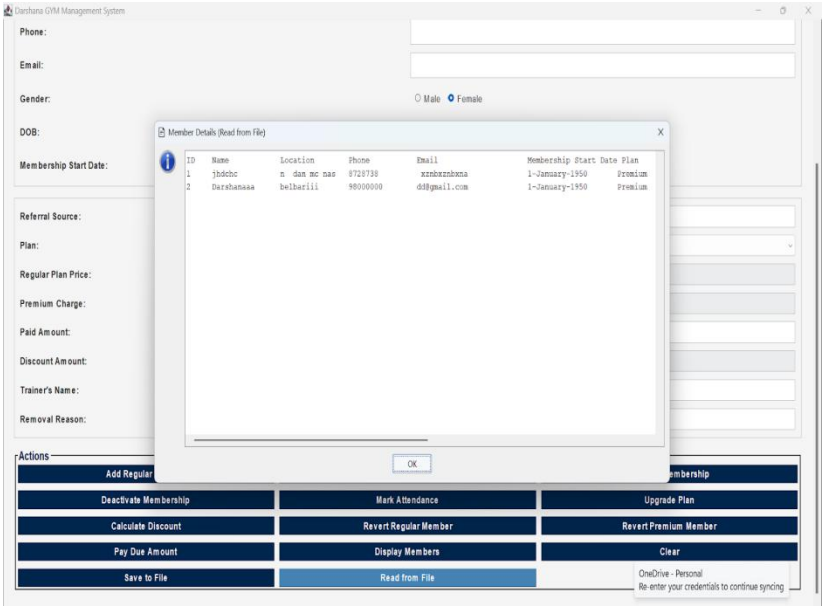


6.5.2 Read From file

Table 15 : Read from File Button Testing

Objective	The readfromfile button is being tested.
Action	First you need to save the member details by clicking the “savetofile” button. This will save member data in “memberDetails.txt” and after that click on the “read from file”button. This will display all the save member details and provide complete information related to each member in the gym management system.
Expected output	It will display the member details
Actual output	It will display all member details like id, name, phonenumber, pay amount, email etc. everything related to gym.
Result	The test was successful.

Figure 20 : Read from file Button successful test.



7 Error Detection and Correction

Error detection refers to the identification of the error that we commit when there is data that we have entered, saved, or sent and rectifying it where applicable. In the management of Gym Systems there are the mistakes that may be committed when the information is keyed in by the user or when files containing information are utilized or being placed. The prevention of any errors that may cause dependence should be carried out through filters and the term of correction. The presence of a gym management app allows one to make sure that the information provided during the entry or transfer activity is correct and does not contain any mistakes. In the case of working with a gym management system this means to verify the information entered by users and see that it is true and accurate. Some employees entering a new member in the gym can incur errors of recording a negative value in age, or incorrect membership type or even a critical field can go unfilled. It is very common when human beings input data. This is done to ensure such errors can be identified and fixed before even saving the data the purpose of the validation is. This is referred to as error detection. Whenever the system finds an error, it must be able to warn the user or solve the error. By doing so, the system would intercept the erroneous term of Premium and substitute it with the appropriate term name of Premium. This can be termed as error correction. After correcting the errors, this system runs very fine and the staff make fewer mistakes and the details split right. Therefore, due to this, it becomes impossible to print incorrect bills, accounts of members will not contain any issues, and the overall system will not crash in case there are no mistakes.

It contributes to maintaining quality and safety of software artefact.

7.1 Syntax Error

If the program is created incorrectly it breaks the language programming principle and results in syntax mistakes. it is unable to understand it similarly to when you use poor language in a statement. For instance with the case of the python a syntax error will occur if.you misspell print or carelessness. To include a period unless you correct this the application won't continue functioning. That kind of error can also occur in a gym system when a wrong symbol is utilized or a period is missing. the program rather the information is the source of these errors.

Figure 21 : Syntax Error Correction & Decatio

```
    if (name.isEmpty() || location.isEmpty() || phone.isEmpty() ||
        email.isEmpty() || (!maleButton.isSelected() && !femaleButton.isSelected())) {
        JOptionPane.showMessageDialog(this, "Please fill all required fields!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    RegularMember member = new RegularMember(id, name, location, phone, email,
                                             gender, dob, startDate, referral);
    members.add(member);

    JOptionPane.showMessageDialog(this, "Regular Member added successfully!", "Success", JOptionPane.INFORMATION_MESSAGE
    clearFields();

} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this, "Please enter a valid ID!", "Error", JOptionPane.ERROR_MESSAGE);
}
```

7.2 Logical Error

In case names of the programs are not properly constructed it will violate the language programming principle and lead to syntax errors. It will not be able to comprehend the language just as it happens to be when you use bad language in a sentence. in this case of an example with the python where a syntax mistake will occur in case you from memory and are negligible mistake entering print. in case you want to enclose a period unless you correct it the application will cease to work. The same thing will happen with a gym system where the wrong symbol or lack of a period has been entered.

Figure 22 : Logical error Correction & Decation.

```
private void markAttendance() {
    try {
        int id = Integer.parseInt(idField.getText());

        for (GymMember member : members) {
            if (member.getId() == id) {
                if (member.getActiveStatus()) {
                    member.markAttendance();
                    JOptionPane.showMessageDialog(this, "Attendance marked successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
                } else {
                    JOptionPane.showMessageDialog(this, "Membership is not active!", "Warning", JOptionPane.WARNING_MESSAGE);
                }
                return;
            }
        }

        JOptionPane.showMessageDialog(this, "Member not found!", "Error", JOptionPane.ERROR_MESSAGE);
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Please enter a valid ID!", "Error", JOptionPane.ERROR_MESSAGE);
    }
}
```

7.3 Runtime Error

The error that occurs during the time of executing some program already being successfully compiled is called a runtime error. There are syntactic errors that can be identified with the help of compilation whereas as a runtime error they can only be identified when a program is running. These errors halt the running of a program, and the program in most cases behaves badly or shuts down. It could be because of poor incorporation of data, inefficient data or improper use of objects and resources.

Figure 23 : Runtime Error Correction & Decation

```
private void addRegularMember() {
    try {
        int id = Integer.parseInt(idField.getText());
        if (isIdDuplicate(id)) {
            JOptionPane.showMessageDialog(null, "Member ID already exists!", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
        // Continue with other logic for adding the member
        JOptionPane.showMessageDialog(null, "Member added successfully!", "Info", JOptionPane.INFORMATION_MESSAGE);
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(null, "Invalid ID. Please enter a valid number.", "Error", JOptionPane.ERROR_MESSAGE);
    }
}
}
```

7. Conclusion

It was a rich experience since the Gym Management System development was in the practical aspect of proceeding to know how to develop the software. Such features of the application as adding or updating and managing the gym members; attendance control, checking user input, permanent saving of the data, and error handling have been successfully implemented.

It is more familiar to me now how to create GUI applications and Java was the library that one could use to develop the GUIs and the code could be maintained and focused to the user. The issues experienced together with the solutions that were adopted assisted me in developing as a developer. This project did not only enable me to apply theoretical knowledge to practice but also demonstrated to me how to think critically, how to debug and do the same with a working solution.

The project can be advanced in future by:

i. To be connected to a database (like MySQL) and enjoy a super-enhanced data store.
ii. The incorporation of administrative logging.
iii. Add a search and filter to be able to control the members and complications
iv. This structure is a steady ground to Java development and this can even be really of assistance in a gym or fitness area.

8.1 Evaluation of Work

The Gym Management System more or less satisfies the objectives of its targeted purpose since it has been in a position to provide an interaction and functional GUI to assist in managing the gym member or the attendance or the history of the gym membership. The regular and premium members are allowed to be entered into the system with the required details such as the id, name, contact details, the status of the membership and partly payment. Significant functions, such as marking attendance, saving and reading files, rollback of membership and validation of inputs, were developed having logical basis and user comments (e.g. -JOptionPane dialogs).

Overall, the system is user friendly and it offers an expedient procedure of managing the information about the gym resourcefully and in a short time. Integrity of data, how to handle errors and easy user experience are provided by the program construction.

8.2 Reflection and Learn

During the process of undertaking this project development, I absorbed the designing and implementation of an actual World Java Swing GUI application.

I gained first hand experience in:

I. Applying the object-oriented ideas of organization codes
II. Creating and collaborating with different consumers (Regular and Premium members).
III. Validation (or checking) of user input which excludes logical and run-time errors.
IV. Reading files and writing into files with Java I/O.
V. Exercise on the exception handling (try-catch) so as to harden the program.

Another area that I improved is control structures, processing of events in a GUI system, and good programming such as modular design and user guidance plans.

8.3 Difficulties Face

During the development of Gym Management System, I encountered the challenges as below:

i. Input validation: It is necessary to be sure that all the required fields were filled correctly before implementing functions like adding members or saving in a file.
ii. Logical flow: Correct nesting of conditions when verifying attendance or revert the members in case they are inactive.
iii. File handling: Reading the information about the members and storing it in file by way of right format and not overwriting others or duplicates.
iv. GUI straightness and design: The fact that the positioning of the elements (text boxes, buttons, labels) was creatively straightened and feasible.

8.4 How Difficulties Were Overcome

A systematic approach to debugging problems which I adopted would result in the elimination of these problems:

i. Exploited the JOptionPane dialog which displayed some meaningful success and error messages so that I would learn during testing.
ii. Ate java docs and other online references to deal with exceptions and correct syntax of Swing components.
iii. Split the program in small convenient functions like clear Fields(), mark Attendance(), saveToFile() and make it more modular and test and repair easy.
iv. Implemented the conditions testing and well-formed statements of returns to make sure that some actions were not executed in case of an error.
v. Using test and adjust each part testing could be done continuously and the logic enhanced thereby making the system user friendly and reliable.

9 References

9.1 file:///C:/Users/sp/Desktop/java.htm

9.2 file:///C:/Users/sp/Desktop/File_BlueJ_Logo.svg

9.3 file:///C:/Users/sp/Desktop/File_Google_Docs_2020_Logo.svg

10. Appendix

10.1 Code Listing for GYM Member

```
package Coursework;


import java.io.Serializable; // Enables object serialization for saving/loading objects


/**
 * Abstract class representing a Gym Member.
 * Implements Serializable to allow objects to be written to and read from a file.
 */
public abstract class GymMember implements Serializable {


    // Serial version ID used during deserialization to verify compatibility
    private static final long serialVersionUID = 1L;


    // Basic member details
    protected int id;
    protected String name;
```

```
    protected String location;
    protected String phone;
    protected String email;
    protected String gender;
    protected String DOB;
    protected String membershipStartDate;

    // Additional fields related to membership
    protected int attendance;
    protected double loyaltyPoints;
    protected boolean activeStatus;

    /**
     * Constructor to initialize a GymMember object.
     *
     * @param id Member ID
     * @param name Full name of the member
     * @param location Address/location of the member
     * @param phone Contact number
     * @param email Email address
     * @param gender Gender of the member
     * @param DOB Date of birth
     * @param membershipStartDate Membership start date
```

```
 */
public GymMember(int id, String name, String location, String phone, String email,
                String gender, String DOB, String membershipStartDate) {
    this.id = id;
    this.name = name;
    this.location = location;
    this.phone = phone;
    this.email = email;
    this.gender = gender;
    this.DOB = DOB;
    this.membershipStartDate = membershipStartDate;
    this.attendance = 0;
    this.loyaltyPoints = 0;
    this.activeStatus = false;
}

// Getter methods (Accessors) to retrieve member information
public int getId() { return id; }
public String getName() { return name; }
public String getLocation() { return location; }
public String getPhone() { return phone; }
public String getEmail() { return email; }
public String getGender() { return gender; }
```

```
    public String getDOB() { return DOB; }

    public String getMembershipStartDate() { return membershipStartDate; }

    public int getAttendance() { return attendance; }

    public double getLoyaltyPoints() { return loyaltyPoints; }

    public boolean getActiveStatus() { return activeStatus; }


    /**
     * Abstract method that must be implemented by subclasses to define how
attendance is marked.
     */
    public abstract void markAttendance();


    /**
     * Activates the member's gym membership.
     */
    public void activateMembership() {
        this.activeStatus = true;
    }


    /**
     * Deactivates the member's gym membership if it's currently active.
     */
    public void deactivateMembership() {
```

```
        if (this.activeStatus) {
            this.activeStatus = false;
        }
    }

    /**
     * Resets the member's attendance and loyalty points and deactivates membership.
     */
    public void resetMember() {
        this.activeStatus = false;
        this.attendance = 0;
        this.loyaltyPoints = 0;
    }

    /**
     * Displays all member details to the console.
     */
    public void display() {
        System.out.println("Member ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Location: " + location);
        System.out.println("Phone: " + phone);
        System.out.println("Email: " + email);
```

```
        System.out.println("Gender: " + gender);
        System.out.println("DOB: " + DOB);
        System.out.println("Membership Start Date: " + membershipStartDate);
        System.out.println("Attendance: " + attendance);
        System.out.println("Loyalty Points: " + loyaltyPoints);
        System.out.println("Active Status: " + activeStatus);
    }
}
```

10.2 code listing for Regular Member

```
package Coursework;

import java.io.Serializable;

/**
 * Represents a Regular Gym Member.
 * Extends GymMember and adds regular-specific attributes and behaviors.
 * Implements Serializable for file-based persistence.
 */
public class RegularMember extends GymMember implements Serializable {
    private static final long serialVersionUID = 1L;
```

```
    // Maximum attendance before becoming eligible for upgrade
    private final int attendanceLimit = 30;

    // Regular member-specific fields
    private boolean isEligibleForUpgrade;  // Tracks if the member can upgrade the plan
    private String removalReason;          // Reason for member removal (if any)
    private String referralSource;         // How the member found the gym (e.g., friend, ad)
    private String plan;                   // Current subscription plan: basic, standard, deluxe
    private double price;                  // Price of the current plan

    /**
     * Constructor to initialize a RegularMember with basic and regular-specific attributes.
     *
     * @param id Member ID
     * @param name Member name
     * @param location Address/location
     * @param phone Contact number
     * @param email Email address
     * @param gender Gender
     * @param DOB Date of birth
     * @param membershipStartDate Membership start date
     * @param referralSource Source from where member heard about the gym
     */
```

```
    public RegularMember(int id, String name, String location, String phone, String email,
                    String gender, String DOB, String membershipStartDate, String
referralSource) {
        super(id, name, location, phone, email, gender, DOB, membershipStartDate);
        this.isEligibleForUpgrade = false;
        this.removalReason = "";
        this.referralSource = referralSource;
        this.plan = "basic";
        this.price = 6500;
    }

    // Getter (accessor) methods for regular-specific attributes
    public int getAttendanceLimit() { return attendanceLimit; }
    public boolean getIsEligibleForUpgrade() { return isEligibleForUpgrade; }
    public String getRemovalReason() { return removalReason; }
    public String getReferralSource() { return referralSource; }
    public String getPlan() { return plan; }
    public double getPrice() { return price; }

    /**
     * Overrides the abstract method to mark attendance.
     * Increments attendance count and adds loyalty points.
     * If attendance reaches the limit, makes member eligible for upgrade.
```

```
     */
    @Override
    public void markAttendance() {
        this.attendance++;
        this.loyaltyPoints += 5;

        // Make member eligible for upgrade if attendance limit is reached
        if (this.attendance >= attendanceLimit) {
            this.isEligibleForUpgrade = true;
        }
    }

    /**
     * Returns the price of a given plan.
     *
     * @param plan The plan name (basic, standard, deluxe)
     * @return The corresponding price or -1 if the plan is invalid
     */
    public double getPlanPrice(String plan) {
        switch (plan.toLowerCase()) {
            case "basic": return 6500;
            case "standard": return 12500;
            case "deluxe": return 18500;
```

```
            default: return -1; // Invalid plan
        }
    }

    /**
     * Allows eligible members to upgrade to a new plan.
     *
     * @param newPlan The plan to upgrade to
     * @return A message showing success or failure reason
     */
    public String upgradePlan(String newPlan) {
        if (!isEligibleForUpgrade) {
            return "Not eligible for upgrade. Need more attendance.";
        }

        if (newPlan.equalsIgnoreCase(this.plan)) {
            return "Already subscribed to this plan.";
        }

        double newPrice = getPlanPrice(newPlan);
        if (newPrice == -1) {
            return "Invalid plan selected.";
        }
```

```
        // Perform upgrade
        this.plan = newPlan.toLowerCase();
        this.price = newPrice;
        return "Plan upgraded successfully to " + newPlan;
    }

    /**
     * Resets this regular member to default state.
     * Stores reason for removal and resets plan and eligibility.
     *
     * @param removalReason Reason why the member is removed or reset
     */
    public void revertRegularMember(String removalReason) {
        super.resetMember(); // Reset common member attributes
        this.isEligibleForUpgrade = false;
        this.plan = "basic";
        this.price = 6500;
        this.removalReason = removalReason;
    }

    /**
     * Displays all details of the regular member.
```

```
     * Includes both base and regular-specific attributes.
     */
    @Override
    public void display() {
        super.display(); // Display base class info
        System.out.println("Plan: " + plan);
        System.out.println("Price: " + price);
        if (!removalReason.isEmpty()) {
            System.out.println("Removal Reason: " + removalReason);
        }
    }
}
```

10.3 code Listing for Premium Member

```
package Coursework;


import java.io.Serializable;


/**
 * Represents a Premium Gym Member.
 * Inherits from the GymMember abstract class and adds premium-specific features.
 * Implements Serializable for object persistence.
 */
```

```
public class PremiumMember extends GymMember implements Serializable {
    private static final long serialVersionUID = 1L;

    // Fixed premium charge for all premium members
    private final double premiumCharge = 50000;

    // Premium-specific attributes
    private String personalTrainer;
    private boolean isFullPayment;
    private double paidAmount;
    private double discountAmount;

    /**
     * Constructor to initialize a PremiumMember with basic and premium attributes.
     *
     * @param id Member ID
     * @param name Full name
     * @param location Address/location of the member
     * @param phone Contact number
     * @param email Email address
     * @param gender Gender
     * @param DOB Date of birth
     * @param membershipStartDate Membership start date
```

```
     * @param personalTrainer Assigned personal trainer
     */
    public PremiumMember(int id, String name, String location, String phone, String email,
                         String gender, String DOB, String membershipStartDate, String personalTrainer) {
        super(id, name, location, phone, email, gender, DOB, membershipStartDate);
        this.personalTrainer = personalTrainer;
        this.isFullPayment = false;
        this.paidAmount = 0.0;
        this.discountAmount = 0.0;
    }

    // Accessor methods for all attributes
    public double getPremiumCharge() {
        return premiumCharge;
    }

    public String getPersonalTrainer() {
        return personalTrainer;
    }

    public boolean getIsFullPayment() {
```

```
        return isFullPayment;
    }

    public double getPaidAmount() {
        return paidAmount;
    }

    public double getDiscountAmount() {
        return discountAmount;
    }

    /**
     * Overrides the abstract markAttendance method.
     * Increases attendance and rewards loyalty points by 5 per visit.
     */
    @Override
    public void markAttendance() {
        this.attendance++;
        this.loyaltyPoints += 5;
    }

    /**
     * Handles payment for the premium membership.
```

```
 *
 * @param paidAmount The amount to be paid in this transaction.
 * @return Message indicating whether the payment was successful or invalid.
 */
public String payDueAmount(double paidAmount) {
    // Case 1: Payment already completed
    if (this.isFullPayment) {
        return "Payment is already complete. No further payment required.";
    }

    // Case 2: Prevent overpayment
    if (this.paidAmount + paidAmount > premiumCharge) {
        return "Payment amount exceeds the premium charge. Maximum payable: " +
                (premiumCharge - this.paidAmount);
    }

    // Case 3: Invalid amount (zero or negative)
    if (paidAmount <= 0) {
        return "Invalid payment amount. Please enter a positive value.";
    }

    // Case 4: Valid payment, update total paid
    this.paidAmount += paidAmount;
```

```
        // Mark payment complete if total paid reaches or exceeds premium charge
        if (this.paidAmount >= premiumCharge) {
            this.isFullPayment = true;
        }

        // Show remaining amount or success message
        double remainingAmount = premiumCharge - this.paidAmount;

        if (this.isFullPayment) {
            return "Payment successful. Payment completed in full!";
        } else {
            return String.format("Payment successful. Remaining amount: %.2f",
remainingAmount);
        }
    }

    /**
     * Calculates discount (10%) if full payment is completed.
     * Displays a message based on eligibility.
     */
    public void calculateDiscount() {
        if (isFullPayment) {
```

```
            this.discountAmount = premiumCharge * 0.10; // 10% discount
            System.out.println("Discount calculated successfully: " +
                                String.format("%.2f", discountAmount));
        } else {
            this.discountAmount = 0.0; // No discount if not fully paid
            System.out.println("No discount available. Complete full payment to receive 10% discount.");
        }
    }

    /**
     * Resets this premium member to default state.
     * Also resets all premium-specific fields in addition to base member fields.
     */
    public void revertPremiumMember() {
        super.resetMember(); // Reset base class fields
        this.personalTrainer = "";
        this.isFullPayment = false;
        this.paidAmount = 0.0;
        this.discountAmount = 0.0;
    }

    /**
```

```
     * Displays all member details, including premium-related data.
     * Calls the base display method and adds premium information.
     */
    @Override
    public void display() {
        super.display(); // Show base details
        System.out.println("Personal Trainer: " +
                        (personalTrainer.isEmpty() ? "Not assigned" : personalTrainer));
        System.out.println("Premium Charge: " + String.format("%.2f", premiumCharge));
        System.out.println("Paid Amount: " + String.format("%.2f", paidAmount));
        System.out.println("Payment Status: " + (isFullPayment ? "Complete" :
"Incomplete"));

        double remainingAmount = premiumCharge - paidAmount;
        System.out.println("Remaining Amount: " + String.format("%.2f",
remainingAmount));

        if (isFullPayment && discountAmount > 0) {
            System.out.println("Discount Amount: " + String.format("%.2f", discountAmount));
        }
    }
}
```

10.4 Code Listing for GYM GUI

```
package Coursework;


import javax.swing.*;
import java.awt.*;
import java.util.ArrayList;
import java.util.Calendar;
import java.io.*;


/**
 * GymGUI - Main GUI class for Darshana Gym Management System
 * This class creates a comprehensive Swing-based interface for managing gym members,
 * including regular and premium members with various operations like adding, updating,
 * file operations, and member management.
 */
public class GymGUI extends JFrame {
    // ================================
```

```
// INSTANCE VARIABLES
// ================================

/** ArrayList to store all gym members (both regular and premium) */
private ArrayList<GymMember> members;

// Personal Information Input Fields
private JTextField idField, nameField, locationField, phoneField, emailField;
private JTextField referralField, paidAmountField, removalReasonField, trainerField;

// Membership Details Input Fields
private JTextField planPriceField, premiumChargeField, discountField;

// Date Selection Components (Birth Date and Membership Start Date)
private JComboBox<String> dobYearComboBox, dobMonthComboBox, dobDayComboBox;
private JComboBox<String> msYearComboBox, msMonthComboBox, msDayComboBox;

// Gender Selection Components
private JRadioButton maleButton, femaleButton;

// Plan Selection Component
```

```
    private JComboBox<String> planComboBox;


    // ================================
    // COLOR SCHEME CONSTANTS
    // ================================
    private Color primaryColor = new Color(0, 38, 77);        // Dark blue for headers
    private Color secondaryColor = new Color(70, 130, 180);    // Steel blue for hover
effects
    private Color accentColor = new Color(239, 134, 134);     // Light red accent (unused
in current code)
    private Color textColor = new Color(50, 50, 50);          // Dark gray for text
    private Color bgColor = new Color(245, 245, 245);         // Light gray background


    /**
     * Constructor - Initializes the GUI and sets up all components
     */
    public GymGUI() {
        members = new ArrayList<>();                          // Initialize member list
        setTitle("Darshana GYM Management System");           // Set window title
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);       // Exit on close
        setupUI();                                            // Create and arrange UI components
        pack();                                               // Size window to fit components
        setLocationRelativeTo(null);                          // Center window on screen
```

```
    }

    /**
     * Sets up all date combo boxes for DOB and membership start date
     * Creates dropdown menus for day (1-31), month (January-December), and year (1950-2024)
     */
    private void setupDateComboBoxes() {
        // Create year options (75 years from 1950 to 2024)
        String[] years = new String[75];
        for (int i = 0; i < 75; i++) {
            years[i] = String.valueOf(1950 + i);
        }

        // Month names array
        String[] months = {"January", "February", "March", "April", "May", "June",
                           "July", "August", "September", "October", "November", "December"};

        // Create day options (1-31)
        String[] days = new String[31];
        for (int i = 0; i < 31; i++) {
            days[i] = String.valueOf(i + 1);
        }
```

```
        // Initialize Date of Birth combo boxes
        dobYearComboBox = new JComboBox<>(years);
        dobMonthComboBox = new JComboBox<>(months);
        dobDayComboBox = new JComboBox<>(days);

        // Initialize Membership Start date combo boxes
        msYearComboBox = new JComboBox<>(years);
        msMonthComboBox = new JComboBox<>(months);
        msDayComboBox = new JComboBox<>(days);

        // Apply consistent styling to all combo boxes
        styleComboBox(dobYearComboBox);
        styleComboBox(dobMonthComboBox);
        styleComboBox(dobDayComboBox);
        styleComboBox(msYearComboBox);
        styleComboBox(msMonthComboBox);
        styleComboBox(msDayComboBox);
    }

    /**
     * Applies consistent styling to combo box components
     * @param comboBox The combo box to style
```

```
     */
    private void styleComboBox(JComboBox<String> comboBox) {
        comboBox.setBackground(Color.WHITE);
        comboBox.setForeground(textColor);
        comboBox.setFont(new Font("Arial", Font.PLAIN, 14));
        ((JComponent) comboBox.getRenderer()).setOpaque(true);
    }

    /**
     * Applies consistent styling to button components with hover effects
     * @param button The button to style
     */
    private void styleButton(JButton button) {
        button.setBackground(primaryColor);
        button.setForeground(Color.WHITE);
        button.setFocusPainted(false);                    // Remove focus border
        button.setFont(new Font("Arial", Font.BOLD, 14));
        button.setBorderPainted(false);                   // Remove default border
        button.setCursor(new  Cursor(Cursor.HAND_CURSOR));        //  Hand  cursor  on
hover

        // Add hover effect listeners
        button.addMouseListener(new java.awt.event.MouseAdapter() {
```

```
            public void mouseEntered(java.awt.event.MouseEvent evt) {
                button.setBackground(secondaryColor);         // Change color on hover
            }

            public void mouseExited(java.awt.event.MouseEvent evt) {
                button.setBackground(primaryColor);           // Restore original color
            }
        });
    }

    /**
     * Applies consistent styling to text field components
     * @param textField The text field to style
     */
    private void styleTextField(JTextField textField) {
        textField.setFont(new Font("Arial", Font.PLAIN, 14));
        textField.setForeground(textColor);
        // Create compound border with line border and padding
        textField.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(189, 195, 199)),
            BorderFactory.createEmptyBorder(5, 7, 5, 7)));
    }
```

```
    /**
     * Applies consistent styling to label components
     * @param label The label to style
     */
    private void styleLabel(JLabel label) {
        label.setFont(new Font("Arial", Font.BOLD, 14));
        label.setForeground(textColor);
    }

    /**
     * Applies consistent styling to radio button components
     * @param radioButton The radio button to style
     */
    private void styleRadioButton(JRadioButton radioButton) {
        radioButton.setFont(new Font("Arial", Font.PLAIN, 14));
        radioButton.setForeground(textColor);
        radioButton.setBackground(bgColor);
    }

    /**
     * Applies consistent styling to panel components
     * @param panel The panel to style
     */
```

```
    private void stylePanel(JPanel panel) {
        panel.setBackground(bgColor);
        // Create compound border with line border and padding
        panel.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(189, 195, 199), 1),
            BorderFactory.createEmptyBorder(10, 10, 10, 10)));
    }

    /**
     * Main method to set up the entire user interface
     * Creates and arranges all GUI components including:
     * - Header panel with title
     * - Personal information input section
     * - Membership details input section
     * - Action buttons panel
     */
    private void setupUI() {
        setLayout(new BorderLayout(10, 10));
        getContentPane().setBackground(bgColor);

        // ================================
        // MAIN CONTAINER PANEL
        // ================================
```

```
JPanel mainPanel = new JPanel();
mainPanel.setLayout(new BoxLayout(mainPanel, BoxLayout.Y_AXIS));
mainPanel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));
mainPanel.setBackground(bgColor);

// ================================
// HEADER PANEL
// ================================
JPanel headerPanel = new JPanel(new BorderLayout());
headerPanel.setBackground(primaryColor);
JLabel titleLabel = new JLabel("Darshana Gym Management System",
JLabel.CENTER);
titleLabel.setFont(new Font("Arial", Font.BOLD, 24));
titleLabel.setForeground(Color.WHITE);
titleLabel.setBorder(BorderFactory.createEmptyBorder(15, 0, 15, 0));
headerPanel.add(titleLabel, BorderLayout.CENTER);

// ================================
// PERSONAL INFORMATION PANEL
// ================================
JPanel personalInfoPanel = new JPanel(new GridLayout(0, 2, 10, 10));
personalInfoPanel.setBorder(BorderFactory.createTitledBorder(
    BorderFactory.createLineBorder(primaryColor, 2),
```

```
        "Personal Information",
        0, 0,
        new Font("Arial", Font.BOLD, 16),
        primaryColor));
    stylePanel(personalInfoPanel);

    // Member ID Field
    JLabel idLabel = new JLabel("Member ID:");
    styleLabel(idLabel);
    personalInfoPanel.add(idLabel);
    idField = new JTextField(10);
    styleTextField(idField);
    personalInfoPanel.add(idField);

    // Name Field
    JLabel nameLabel = new JLabel("Name:");
    styleLabel(nameLabel);
    personalInfoPanel.add(nameLabel);
    nameField = new JTextField(20);
    styleTextField(nameField);
    personalInfoPanel.add(nameField);

    // Location Field
```

```
JLabel locationLabel = new JLabel("Location:");
styleLabel(locationLabel);
personalInfoPanel.add(locationLabel);
locationField = new JTextField(20);
styleTextField(locationField);
personalInfoPanel.add(locationField);

// Phone Field
JLabel phoneLabel = new JLabel("Phone:");
styleLabel(phoneLabel);
personalInfoPanel.add(phoneLabel);
phoneField = new JTextField(15);
styleTextField(phoneField);
personalInfoPanel.add(phoneField);

// Email Field
JLabel emailLabel = new JLabel("Email:");
styleLabel(emailLabel);
personalInfoPanel.add(emailLabel);
emailField = new JTextField(20);
styleTextField(emailField);
personalInfoPanel.add(emailField);
```

```
// Gender Selection (Radio Buttons)
JLabel genderLabel = new JLabel("Gender:");
styleLabel(genderLabel);
personalInfoPanel.add(genderLabel);
JPanel genderPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
genderPanel.setBackground(bgColor);

ButtonGroup genderGroup = new ButtonGroup();  // Ensures only one gender can be selected
maleButton = new JRadioButton("Male");
femaleButton = new JRadioButton("Female");
styleRadioButton(maleButton);
styleRadioButton(femaleButton);
genderGroup.add(maleButton);
genderGroup.add(femaleButton);
genderPanel.add(maleButton);
genderPanel.add(femaleButton);
personalInfoPanel.add(genderPanel);

// Initialize date combo boxes
setupDateComboBoxes();

// Date of Birth Selection
```

```
JLabel dobLabel = new JLabel("DOB:");
styleLabel(dobLabel);
personalInfoPanel.add(dobLabel);
JPanel dobPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
dobPanel.setBackground(bgColor);
dobPanel.add(dobDayComboBox);
dobPanel.add(dobMonthComboBox);
dobPanel.add(dobYearComboBox);
personalInfoPanel.add(dobPanel);

// Membership Start Date Selection
JLabel msLabel = new JLabel("Membership Start Date:");
styleLabel(msLabel);
personalInfoPanel.add(msLabel);
JPanel msPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
msPanel.setBackground(bgColor);
msPanel.add(msDayComboBox);
msPanel.add(msMonthComboBox);
msPanel.add(msYearComboBox);
personalInfoPanel.add(msPanel);

// ================================
// MEMBERSHIP DETAILS PANEL
```

```
// ================================
JPanel membershipPanel = new JPanel(new GridLayout(0, 2, 10, 10));
membershipPanel.setBorder(BorderFactory.createTitledBorder(
    BorderFactory.createLineBorder(primaryColor, 2),
    "Membership Details",
    0, 0,
    new Font("Arial", Font.BOLD, 16),
    primaryColor));
stylePanel(membershipPanel);

// Referral Source Field
JLabel referralLabel = new JLabel("Referral Source:");
styleLabel(referralLabel);
membershipPanel.add(referralLabel);
referralField = new JTextField(20);
styleTextField(referralField);
membershipPanel.add(referralField);

// Plan Selection Combo Box
JLabel planLabel = new JLabel("Plan:");
styleLabel(planLabel);
membershipPanel.add(planLabel);
planComboBox = new JComboBox<>(new String[]{"Basic", "Standard", "Deluxe"});
```

```
        styleComboBox(planComboBox);
        membershipPanel.add(planComboBox);

        // Plan combo box event listener - automatically updates price based on selected plan
        planComboBox.addActionListener(e -> {
            String selectedPlan = (String) planComboBox.getSelectedItem();
            switch (selectedPlan.toLowerCase()) {
                case "basic":
                    planPriceField.setText("6500");
                    break;
                case "standard":
                    planPriceField.setText("12500");
                    break;
                case "deluxe":
                    planPriceField.setText("18500");
                    break;
            }
        });

        // Regular Plan Price Field (Read-only, auto-populated)
        JLabel planPriceLabel = new JLabel("Regular Plan Price:");
        styleLabel(planPriceLabel);
```

```
membershipPanel.add(planPriceLabel);
planPriceField = new JTextField("6500");
planPriceField.setEditable(false);  // Cannot be edited by user
styleTextField(planPriceField);
planPriceField.setBackground(new Color(234, 236, 238));  // Gray background for read-only
membershipPanel.add(planPriceField);

// Premium Charge Field (Read-only, fixed value)
JLabel premiumChargeLabel = new JLabel("Premium Charge:");
styleLabel(premiumChargeLabel);
membershipPanel.add(premiumChargeLabel);
premiumChargeField = new JTextField("50000");
premiumChargeField.setEditable(false);  // Fixed premium charge
styleTextField(premiumChargeField);
premiumChargeField.setBackground(new Color(234, 236, 238));
membershipPanel.add(premiumChargeField);

// Paid Amount Field
JLabel paidAmountLabel = new JLabel("Paid Amount:");
styleLabel(paidAmountLabel);
membershipPanel.add(paidAmountLabel);
paidAmountField = new JTextField(10);
```

```
styleTextField(paidAmountField);
membershipPanel.add(paidAmountField);

// Discount Amount Field (Read-only, calculated automatically)
JLabel discountLabel = new JLabel("Discount Amount:");
styleLabel(discountLabel);
membershipPanel.add(discountLabel);
discountField = new JTextField("0");
discountField.setEditable(false);  // Calculated automatically
styleTextField(discountField);
discountField.setBackground(new Color(234, 236, 238));
membershipPanel.add(discountField);

// Trainer's Name Field (for premium members)
JLabel trainerLabel = new JLabel("Trainer's Name:");
styleLabel(trainerLabel);
membershipPanel.add(trainerLabel);
trainerField = new JTextField(20);
styleTextField(trainerField);
membershipPanel.add(trainerField);

// Removal Reason Field (for member removal)
JLabel removalReasonLabel = new JLabel("Removal Reason:");
```

```
styleLabel(removalReasonLabel);
membershipPanel.add(removalReasonLabel);
removalReasonField = new JTextField(20);
styleTextField(removalReasonField);
membershipPanel.add(removalReasonField);

// ==============================
// ACTION BUTTONS PANEL
// ==============================
JPanel buttonPanel = new JPanel(new GridLayout(0, 3, 10, 10));
buttonPanel.setBorder(BorderFactory.createTitledBorder(
    BorderFactory.createLineBorder(primaryColor, 2),
    "Actions",
    0, 0,
    new Font("Arial", Font.BOLD, 16),
    primaryColor));
buttonPanel.setBackground(bgColor);

// Create all action buttons
JButton addRegularButton = new JButton("Add Regular Member");
JButton addPremiumButton = new JButton("Add Premium Member");
JButton activateButton = new JButton("Activate Membership");
JButton deactivateButton = new JButton("Deactivate Membership");
```

```
JButton markAttendanceButton = new JButton("Mark Attendance");
JButton upgradePlanButton = new JButton("Upgrade Plan");
JButton calculateDiscountButton = new JButton("Calculate Discount");
JButton revertRegularButton = new JButton("Revert Regular Member");
JButton revertPremiumButton = new JButton("Revert Premium Member");
JButton payDueButton = new JButton("Pay Due Amount");
JButton displayButton = new JButton("Display Members");
JButton clearButton = new JButton("Clear");

// File Operation Buttons
JButton saveToFileButton = new JButton("Save to File");
JButton readFromFileButton = new JButton("Read from File");

// Apply styling to all buttons
styleButton(addRegularButton);
styleButton(addPremiumButton);
styleButton(activateButton);
styleButton(deactivateButton);
styleButton(markAttendanceButton);
styleButton(upgradePlanButton);
styleButton(calculateDiscountButton);
styleButton(revertRegularButton);
styleButton(revertPremiumButton);
```

```
        styleButton(payDueButton);
        styleButton(displayButton);
        styleButton(clearButton);
        styleButton(saveToFileButton);
        styleButton(readFromFileButton);

        // Special green color for certain buttons
        Color greenColor = new Color(46, 125, 50);
        addRegularButton.setBackground(greenColor);
        addPremiumButton.setBackground(greenColor);
        displayButton.setBackground(greenColor);
        saveToFileButton.setBackground(greenColor);
        readFromFileButton.setBackground(greenColor);

        // Add all buttons to the button panel
        buttonPanel.add(addRegularButton);
        buttonPanel.add(addPremiumButton);
        buttonPanel.add(activateButton);
        buttonPanel.add(deactivateButton);
        buttonPanel.add(markAttendanceButton);
        buttonPanel.add(upgradePlanButton);
        buttonPanel.add(calculateDiscountButton);
        buttonPanel.add(revertRegularButton);
```

```
buttonPanel.add(revertPremiumButton);
buttonPanel.add(payDueButton);
buttonPanel.add(displayButton);
buttonPanel.add(clearButton);
buttonPanel.add(saveToFileButton);
buttonPanel.add(readFromFileButton);

// ================================
// ASSEMBLE MAIN LAYOUT
// ================================
mainPanel.add(headerPanel);
mainPanel.add(Box.createRigidArea(new Dimension(0, 15)));  // Spacing
mainPanel.add(personalInfoPanel);
mainPanel.add(Box.createRigidArea(new Dimension(0, 15)));  // Spacing
mainPanel.add(membershipPanel);
mainPanel.add(Box.createRigidArea(new Dimension(0, 15)));  // Spacing
mainPanel.add(buttonPanel);

// Add main panel to scrollable container
add(new JScrollPane(mainPanel), BorderLayout.CENTER);

// Set preferred window size
setPreferredSize(new Dimension(1100, 800));
```

```
        // ================================
        // BUTTON EVENT HANDLERS
        // ================================
        addRegularButton.addActionListener(e -> addRegularMember());
        addPremiumButton.addActionListener(e -> addPremiumMember());
        activateButton.addActionListener(e -> activateMembership());
        deactivateButton.addActionListener(e -> deactivateMembership());
        markAttendanceButton.addActionListener(e -> markAttendance());
        upgradePlanButton.addActionListener(e -> upgradePlan());
        calculateDiscountButton.addActionListener(e -> calculateDiscount());
        revertRegularButton.addActionListener(e -> revertRegularMember());
        revertPremiumButton.addActionListener(e -> revertPremiumMember());
        payDueButton.addActionListener(e -> payDueAmount());
        displayButton.addActionListener(e -> displayMembers());
        clearButton.addActionListener(e -> clearFields());

        // File operation event handlers
        saveToFileButton.addActionListener(e -> saveToFile());
        readFromFileButton.addActionListener(e -> readFromFile());
    }

    // ================================
```

```
// FILE OPERATION METHODS
// ================================


/**
 * Saves all member data to a text file called "MemberDetails.txt"
 * Creates a formatted table with headers and member information
 */
private void saveToFile() {
    try (PrintWriter writer = new PrintWriter(new FileWriter("MemberDetails.txt"))) {
        // Write table header as specified in requirements
        writer.printf("%-5s %-15s %-15s %-15s %-25s %-20s %-10s %-10s %-10s %-
15s %-10s %-15s %-15s %-15s\n",
            "ID", "Name", "Location", "Phone", "Email", "Membership Start Date",
            "Plan", "Price", "Attendance", "Loyalty Points", "Active Status",
            "Full Payment", "Discount Amount", "Net Amount Paid");


        // Write data for each member
        for (GymMember member : members) {
            String plan = "";
            String price = "";
            String fullPayment = "";
            String discountAmount = "";
            String netAmountPaid = "";
```

```
        // Handle different member types (Regular vs Premium)
        if (member instanceof RegularMember) {
            RegularMember regMember = (RegularMember) member;
            plan = regMember.getPlan();
            price = String.valueOf(regMember.getPrice());
            fullPayment = "N/A";        // Not applicable for regular members
            discountAmount = "N/A";     // Not applicable for regular members
            netAmountPaid = "N/A";      // Not applicable for regular members
        } else if (member instanceof PremiumMember) {
            PremiumMember premMember = (PremiumMember) member;
            plan = "Premium";
            price = String.valueOf(premMember.getPremiumCharge());
            fullPayment = premMember.getIsFullPayment() ? "Yes" : "No";
            discountAmount = String.valueOf(premMember.getDiscountAmount());
            netAmountPaid = String.valueOf(premMember.getPaidAmount());
        }

        // Write formatted member data
        writer.printf("%-5d %-15s %-15s %-15s %-25s %-20s %-10s %-10s %-10d %-
15.1f %-10s %-15s %-15s %-15s\n",
            member.getId(),
            member.getName(),
```

```
                member.getLocation(),
                member.getPhone(),
                member.getEmail(),
                member.getMembershipStartDate(),
                plan,
                price,
                member.getAttendance(),
                member.getLoyaltyPoints(),
                member.getActiveStatus() ? "Active" : "Inactive",
                fullPayment,
                discountAmount,
                netAmountPaid);
            }

            // Show success message
            JOptionPane.showMessageDialog(this, "Data saved to MemberDetails.txt successfully!",
                "Save Successful", JOptionPane.INFORMATION_MESSAGE);

        } catch (IOException e) {
            // Show error message if file operation fails
            JOptionPane.showMessageDialog(this, "Error saving file: " + e.getMessage(),
                "Save Error", JOptionPane.ERROR_MESSAGE);
```

```
        }
    }

    /**
     * Reads member data from "MemberDetails.txt" file and displays it in a dialog
     * Shows the content in a scrollable text area with monospaced font for proper alignment
     */
    private void readFromFile() {
        File file = new File("MemberDetails.txt");

        // Check if file exists
        if (!file.exists()) {
            JOptionPane.showMessageDialog(this, "File 'MemberDetails.txt' does not exist!",
                    "File Not Found", JOptionPane.ERROR_MESSAGE);
            return;
        }

        try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
            StringBuilder fileContent = new StringBuilder();
            String line;

            // Read all lines from file
```

```
        while ((line = reader.readLine()) != null) {
            fileContent.append(line).append("\n");
        }

        // Create text area with monospaced font for proper table alignment
        JTextArea textArea = new JTextArea(fileContent.toString());
        textArea.setFont(new java.awt.Font("Monospaced", java.awt.Font.PLAIN, 13));
        textArea.setEditable(false);    // Read-only
        textArea.setCaretPosition(0);   // Scroll to top

        // Put text area in scrollable container
        JScrollPane scrollPane = new JScrollPane(textArea);
        scrollPane.setPreferredSize(new Dimension(900, 400));

scrollPane.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);

scrollPane.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLLBAR_AS_
NEEDED);

        // Show in dialog box
        JOptionPane.showMessageDialog(this, scrollPane, "    Member Details (Read
from File)",
                JOptionPane.INFORMATION_MESSAGE);
```

```
        } catch (IOException e) {
            // Show error message if file reading fails
            JOptionPane.showMessageDialog(this, "Error reading file: " + e.getMessage(),
                    "Read Error", JOptionPane.ERROR_MESSAGE);
        }
    }

    // ================================
    // MEMBER MANAGEMENT METHODS
    // ================================

    /**
     * Adds a new regular member to the system
     * Validates input fields and checks for duplicate IDs
     */
    private void addRegularMember() {
        try {
            // Parse and validate member ID
            int id = Integer.parseInt(idField.getText());
            if (isIdDuplicate(id)) {
                JOptionPane.showMessageDialog(this, "Member ID already exists!", "Error",
JOptionPane.ERROR_MESSAGE);
                return;
```

```
        }

        // Get input values
        String name = nameField.getText();
        String location = locationField.getText();
        String phone = phoneField.getText();
        String email = emailField.getText();
        String gender = maleButton.isSelected() ? "Male" : "Female";

        // Format date strings from combo box selections
        String dob = dobDayComboBox.getSelectedItem() + "-" +
                dobMonthComboBox.getSelectedItem() + "-" +
                dobYearComboBox.getSelectedItem();
        String startDate = msDayComboBox.getSelectedItem() + "-" +
                    msMonthComboBox.getSelectedItem() + "-" +
                    msYearComboBox.getSelectedItem();
        String referral = referralField.getText();

        // Validate required fields
        if (name.isEmpty() || location.isEmpty() || phone.isEmpty() ||
            email.isEmpty() || (!maleButton.isSelected() && !femaleButton.isSelected())) {
            JOptionPane.showMessageDialog(this, "Please fill all required fields!", "Error",
JOptionPane.ERROR_MESSAGE);
```

```
            return;
        }

        // Create new regular member and add to list
        RegularMember member = new RegularMember(id, name, location, phone, email,
                                gender, dob, startDate, referral);
        members.add(member);

        // Show success message and clear form
        JOptionPane.showMessageDialog(this, "Regular Member added successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
        clearFields();

    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Please enter a valid ID!", "Error", JOptionPane.ERROR_MESSAGE);
    }
}

/**
 * Adds a new premium member to the system
 * Similar to addRegularMember but creates PremiumMember object
```

```
    */
   private void addPremiumMember() {
      try {
         // Parse and validate member ID
         int id = Integer.parseInt(idField.getText());
         if (isIdDuplicate(id)) {
            JOptionPane.showMessageDialog(this, "Member ID already exists!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
         }

         // Get input values
         String name = nameField.getText();
         String location = locationField.getText();
         String phone = phoneField.getText();
         String email = emailField.getText();
         String gender = maleButton.isSelected() ? "Male" : "Female";

         // Format date strings
         String dob = dobDayComboBox.getSelectedItem() + "-" +
                  dobMonthComboBox.getSelectedItem() + "-" +
                  dobYearComboBox.getSelectedItem();
         String startDate = msDayComboBox.getSelectedItem() + "-" +
```

```
                msMonthComboBox.getSelectedItem() + "-" +
                msYearComboBox.getSelectedItem();
        String trainer = trainerField.getText();

        // Validate required fields
        if (name.isEmpty() || location.isEmpty() || phone.isEmpty() ||
            email.isEmpty() || (!maleButton.isSelected() && !femaleButton.isSelected())) {
            JOptionPane.showMessageDialog(this, "Please fill all required fields!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Create new premium member and add to list
        PremiumMember member = new PremiumMember(id, name, location, phone,
email,
                                gender, dob, startDate, trainer);
        members.add(member);

        // Show success message and clear form
        JOptionPane.showMessageDialog(this, "Premium Member added successfully!",
"Success", JOptionPane.INFORMATION_MESSAGE);
        clearFields();
```

```
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Please enter a valid ID!", "Error",
JOptionPane.ERROR_MESSAGE);
        }
    }

    /**
     * Checks if a member ID already exists in the system
     * @param id The ID to check
     * @return true if ID exists, false otherwise
     */
    private boolean isIdDuplicate(int id) {
        for (GymMember member : members) {
            if (member.getId() == id) {
                return true;
            }
        }
        return false;
    }

    /**
     * Activates membership for a member with the specified ID
     * Searches for member by ID and calls their activateMembership method
```

```
     */
    private void activateMembership() {
        try {
            int id = Integer.parseInt(idField.getText());


            // Search for member with matching ID
            for (GymMember member : members) {
                if (member.getId() == id) {
                    member.activateMembership();
                    JOptionPane.showMessageDialog(this, "Membership activated successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
                    return;
                }
            }


            // Member not found
            JOptionPane.showMessageDialog(this, "Member not found!", "Error", JOptionPane.ERROR_MESSAGE);
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Please enter a valid ID!", "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
```

```
/**
 * Deactivates membership for a member with the specified ID
 * Searches for member by ID and calls their deactivateMembership method
 */
private void deactivateMembership() {
  try {
      int id = Integer.parseInt(idField.getText());

      // Search for member with matching ID
      for (GymMember member : members) {
          if (member.getId() == id) {
              member.deactivateMembership();
              JOptionPane.showMessageDialog(this, "Membership deactivated successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
              return;
          }
      }

      // Member not found
      JOptionPane.showMessageDialog(this, "Member not found!", "Error", JOptionPane.ERROR_MESSAGE);
  } catch (NumberFormatException e) {
```

```
        JOptionPane.showMessageDialog(this, "Please enter a valid ID!", "Error", JOptionPane.ERROR_MESSAGE);
    }
  }

  /**
   * Marks attendance for a member with the specified ID
   * Only allows attendance marking if membership is active
   */
  private void markAttendance() {
      try {
          int id = Integer.parseInt(idField.getText());

          // Search for member with matching ID
          for (GymMember member : members) {
              if (member.getId() == id) {
                  // Check if membership is active before marking attendance
                  if (member.getActiveStatus()) {
                      member.markAttendance();
                      JOptionPane.showMessageDialog(this, "Attendance marked successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
                  } else {
```

```
                JOptionPane.showMessageDialog(this, "Membership is not active!", "Warning", JOptionPane.WARNING_MESSAGE);
            }
            return;
        }
    }


    // Member not found
    JOptionPane.showMessageDialog(this, "Member not found!", "Error", JOptionPane.ERROR_MESSAGE);
  } catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this, "Please enter a valid ID!", "Error", JOptionPane.ERROR_MESSAGE);
  }
}


/**
 * Upgrades the plan for a regular member
 * Only works for regular members with active membership
 */
private void upgradePlan() {
  try {
    int id = Integer.parseInt(idField.getText());
```

```
            String selectedPlan = (String) planComboBox.getSelectedItem();


            // Search for member with matching ID
for (GymMember member : members) {
    if (member.getId() == id) {
        // Check if member is a regular member
        if (member instanceof RegularMember) {
            RegularMember regularMember = (RegularMember) member;
            // Check if membership is active
            if (member.getActiveStatus()) {
                String result = regularMember.upgradePlan(selectedPlan);
                JOptionPane.showMessageDialog(this, result, "Plan Upgrade",
JOptionPane.INFORMATION_MESSAGE);
            } else {
                JOptionPane.showMessageDialog(this, "Membership is not active!",
"Warning", JOptionPane.WARNING_MESSAGE);
            }
        } else {
            JOptionPane.showMessageDialog(this, "Not a Regular Member!", "Error",
JOptionPane.ERROR_MESSAGE);
        }
        return;
    }
```

```
}
            // Member not found
            JOptionPane.showMessageDialog(this, "Member not found!", "Error", JOptionPane.ERROR_MESSAGE);
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Please enter a valid ID!", "Error", JOptionPane.ERROR_MESSAGE);
        }
    }

    /**
     * Calculates discount for a premium member based on their loyalty points
     * Only works for premium members, updates the discount field with calculated amount
     */
    private void calculateDiscount() {
        try {
            int id = Integer.parseInt(idField.getText());

            // Search for member with matching ID
            for (GymMember member : members) {
                if (member.getId() == id) {
                    // Check if member is a premium member
```

```
                if (member instanceof PremiumMember) {
                    PremiumMember premiumMember = (PremiumMember) member;
                    premiumMember.calculateDiscount();
                    // Update discount field with calculated amount

discountField.setText(String.valueOf(premiumMember.getDiscountAmount()));
                    JOptionPane.showMessageDialog(this, "Discount calculated!", "Success", JOptionPane.INFORMATION_MESSAGE);
                } else {
                    JOptionPane.showMessageDialog(this, "Not a Premium Member!", "Error", JOptionPane.ERROR_MESSAGE);
                }
                return;
            }
        }

        // Member not found
        JOptionPane.showMessageDialog(this, "Member not found!", "Error", JOptionPane.ERROR_MESSAGE);
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Please enter a valid ID!", "Error", JOptionPane.ERROR_MESSAGE);
    }
}
```

```
/**
 * Reverts (removes) a regular member from the system
 * Requires a removal reason to be specified
 */
private void revertRegularMember() {
    try {
        int id = Integer.parseInt(idField.getText());
        String reason = removalReasonField.getText();

        // Validate removal reason is provided
        if (reason.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Please enter a removal reason!",
"Warning", JOptionPane.WARNING_MESSAGE);
            return;
        }

        // Search for member with matching ID
        for (GymMember member : members) {
            if (member.getId() == id) {
                // Check if member is a regular member
                if (member instanceof RegularMember) {
                    RegularMember regularMember = (RegularMember) member;
```

```
                    regularMember.revertRegularMember(reason);
                    JOptionPane.showMessageDialog(this, "Regular Member reverted successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
                } else {
                    JOptionPane.showMessageDialog(this, "Not a Regular Member!", "Error", JOptionPane.ERROR_MESSAGE);
                }
                return;
            }
        }

        // Member not found
        JOptionPane.showMessageDialog(this, "Member not found!", "Error", JOptionPane.ERROR_MESSAGE);
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Please enter a valid ID!", "Error", JOptionPane.ERROR_MESSAGE);
    }
}

/**
 * Reverts (removes) a premium member from the system
 * No removal reason required for premium members
 */
```

```
    private void revertPremiumMember() {
        try {
            int id = Integer.parseInt(idField.getText());


            // Search for member with matching ID
            for (GymMember member : members) {
                if (member.getId() == id) {
                    // Check if member is a premium member
                    if (member instanceof PremiumMember) {
                        PremiumMember premiumMember = (PremiumMember) member;
                        premiumMember.revertPremiumMember();
                        JOptionPane.showMessageDialog(this, "Premium Member reverted
successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
                    } else {
                        JOptionPane.showMessageDialog(this, "Not a Premium Member!",
"Error", JOptionPane.ERROR_MESSAGE);
                    }
                    return;
                }
            }


            // Member not found
```

```
            JOptionPane.showMessageDialog(this, "Member not found!", "Error", JOptionPane.ERROR_MESSAGE);
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Please enter a valid ID!", "Error", JOptionPane.ERROR_MESSAGE);
        }
    }

    /**
     * Processes payment for a premium member's due amount
     * Updates the member's payment status and calculates remaining balance
     */
    private void payDueAmount() {
        try {
            int id = Integer.parseInt(idField.getText());
            double amount = Double.parseDouble(paidAmountField.getText());

            // Search for member with matching ID
            for (GymMember member : members) {
                if (member.getId() == id) {
                    // Check if member is a premium member
                    if (member instanceof PremiumMember) {
                        PremiumMember premiumMember = (PremiumMember) member;
```

```
                String result = premiumMember.payDueAmount(amount);
                JOptionPane.showMessageDialog(this, result, "Payment", JOptionPane.INFORMATION_MESSAGE);
            } else {
                JOptionPane.showMessageDialog(this, "Not a Premium Member!", "Error", JOptionPane.ERROR_MESSAGE);
            }
            return;
        }
    }

    // Member not found
    JOptionPane.showMessageDialog(this, "Member not found!", "Error", JOptionPane.ERROR_MESSAGE);
  } catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this, "Please enter valid numbers!", "Error", JOptionPane.ERROR_MESSAGE);
  }
}

/**
 * Displays all members in a new window with formatted information
 * Shows different layouts for regular vs premium members
 * Creates a scrollable dialog with all member details
```

```
     */
    private void displayMembers() {
        // Create new window for displaying members
        JFrame displayFrame = new JFrame("Member Details");
        displayFrame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        // Create text area for member information
        JTextArea textArea = new JTextArea(20, 40);
        textArea.setEditable(false);                    // Read-only
        textArea.setFont(new Font("Monospaced", Font.PLAIN, 14));      // Monospaced for
alignment
        textArea.setBackground(new Color(250, 250, 250));
        textArea.setForeground(textColor);

        // Add scroll capability
        JScrollPane scrollPane = new JScrollPane(textArea);
        scrollPane.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

        // Check if there are any members to display
        if (members.isEmpty()) {
            textArea.setText("No members found!");
        } else {
            StringBuilder displayText = new StringBuilder();
```

```
        // Loop through all members and format their information
        for (GymMember member : members) {
            // Add header based on member type
            if (member instanceof RegularMember) {

displayText.append("╔═══════════════════════════════════════╗\n");
                displayText.append("│       REGULAR MEMBER        │\n");

displayText.append("╚═══════════════════════════════════════╝\n\n");
            } else if (member instanceof PremiumMember) {

displayText.append("╔═══════════════════════════════════════╗\n");
                displayText.append("│       PREMIUM MEMBER        │\n");

displayText.append("╚═══════════════════════════════════════╝\n\n");
            }

            // Add common member information
            displayText.append("ID: ").append(member.getId()).append("\n");
            displayText.append("Name: ").append(member.getName()).append("\n");
            displayText.append("Location: ").append(member.getLocation()).append("\n");
            displayText.append("Phone: ").append(member.getPhone()).append("\n");
            displayText.append("Email: ").append(member.getEmail()).append("\n");
```

```
            displayText.append("Gender: ").append(member.getGender()).append("\n");

            displayText.append("DOB: ").append(member.getDOB()).append("\n");

            displayText.append("Start Date: ").append(member.getMembershipStartDate()).append("\n");

            displayText.append("Attendance: ").append(member.getAttendance()).append("\n");

            displayText.append("Loyalty Points: ").append(member.getLoyaltyPoints()).append("\n");

            displayText.append("Active Status: ").append(member.getActiveStatus() ? "Active" : "Inactive").append("\n");


            // Add type-specific information

            if (member instanceof RegularMember) {

                RegularMember regMember = (RegularMember) member;

                displayText.append("Plan: ").append(regMember.getPlan()).append("\n");

                displayText.append("Price: ").append(regMember.getPrice()).append("\n");

                // Show removal reason if member was removed

                if (!regMember.getRemovalReason().isEmpty()) {

                    displayText.append("Removal Reason: ").append(regMember.getRemovalReason()).append("\n");

                }

            } else if (member instanceof PremiumMember) {

                PremiumMember premMember = (PremiumMember) member;
```

```
            displayText.append("Personal Trainer: ").append(premMember.getPersonalTrainer()).append("\n");

            displayText.append("Paid Amount: ").append(premMember.getPaidAmount()).append("\n");

            displayText.append("Payment Status: ").append(premMember.getIsFullPayment() ? "Complete" : "Incomplete").append("\n");

            // Calculate and show remaining amount

            double remainingAmount = premMember.getPremiumCharge() - premMember.getPaidAmount();

            displayText.append("Remaining Amount: ").append(remainingAmount).append("\n");


            // Show discount if payment is complete

            if (premMember.getIsFullPayment()) {

                displayText.append("Discount Amount: ").append(premMember.getDiscountAmount()).append("\n");

            }

        }


        // Add separator between members

displayText.append("\n═══════════════════════════════════════════════════════════════\n\n");

    }
```

```
        textArea.setText(displayText.toString());
    }

    // Set up display window layout
    displayFrame.add(scrollPane);
    displayFrame.setSize(600, 600);
    displayFrame.setLocationRelativeTo(this);  // Center relative to main window

    // Add header panel to display window
    JPanel headerPanel = new JPanel(new BorderLayout());
    headerPanel.setBackground(primaryColor);
    JLabel titleLabel = new JLabel("Member Details", JLabel.CENTER);
    titleLabel.setFont(new Font("Arial", Font.BOLD, 18));
    titleLabel.setForeground(Color.WHITE);
    titleLabel.setBorder(BorderFactory.createEmptyBorder(10, 0, 10, 0));
    headerPanel.add(titleLabel, BorderLayout.CENTER);

    displayFrame.add(headerPanel, BorderLayout.NORTH);
    displayFrame.add(scrollPane, BorderLayout.CENTER);

    // Add close button at bottom
    JPanel buttonPanel = new JPanel();
    buttonPanel.setBackground(bgColor);
```

```
        JButton closeButton = new JButton("Close");
        styleButton(closeButton);
        closeButton.addActionListener(e -> displayFrame.dispose());  // Close window when clicked
        buttonPanel.add(closeButton);
        displayFrame.add(buttonPanel, BorderLayout.SOUTH);

        // Show the display window
        displayFrame.setVisible(true);
    }

    /**
     * Clears all input fields and resets form to default state
     * Resets text fields, combo boxes, and radio buttons
     */
    private void clearFields() {
        // Clear all text fields
        idField.setText("");
        nameField.setText("");
        locationField.setText("");
        phoneField.setText("");
        emailField.setText("");
        referralField.setText("");
```

```
        paidAmountField.setText("");
        removalReasonField.setText("");
        trainerField.setText("");
        discountField.setText("0");

        // Reset plan combo box to first option (Basic)
        planComboBox.setSelectedIndex(0);

        // Reset date combo boxes to first options
        dobDayComboBox.setSelectedIndex(0);
        dobMonthComboBox.setSelectedIndex(0);
        dobYearComboBox.setSelectedIndex(0);

        msDayComboBox.setSelectedIndex(0);
        msMonthComboBox.setSelectedIndex(0);
        msYearComboBox.setSelectedIndex(0);

        // Clear gender selection
        maleButton.setSelected(false);
        femaleButton.setSelected(false);
    }

    // ================================
```

```
    // MAIN METHOD
    // ================================


    /**
     * Main method to start the application
     * Sets system look and feel and creates the main GUI window
     * @param args Command line arguments (not used)
     */
    public static void main(String[] args) {
        try {
            // Set system look and feel for native appearance
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception e) {
            e.printStackTrace();
        }


        // Create and show GUI on Event Dispatch Thread
        SwingUtilities.invokeLater(() -> {
            new GymGUI().setVisible(true);
        });
    }
}
```